

TalkWith

Proyecto Integrador · Entrega 3

APIs y formularios

Tecnologías de Desarrollo en el Cliente

Diego Gómez · Isaian Ayala · Carlos Olmedo

Noviembre de 2026

Contenido

1. Descripción del proyecto 3
Stack tecnológico 3
Cambios en esta entrega 3
2. Arquitectura del frontend 5
Rutas de la aplicación 5
3. Datos, formularios y manejo de errores 7
3.1 Fuente de datos: API simulada en la capa de servicios 7
3.2 Capa de servicios y manejo de errores 7
3.3 Contrato de la API 7
3.4 Reglas de validación 9
3.5 Estados de la interfaz 9
4. Secciones de la aplicación 11
4.1 Página pública 11
4.2 Pantallas de autenticación de usuario 12
4.3 Inicio y servidores 16
4.4 Canal de texto 21
4.5 Canal de voz y grabación 22
4.6 Historial y grabaciones 25
4.7 Búsqueda global 28
4.8 Configuración del servidor y CRUD de canales 29
4.9 Cuenta 36
5. Mensajes de confirmación, notificaciones y errores 38
6. Repositorio del proyecto 39
Reparto de pantallas y tareas 39

Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».

1. Descripción del proyecto

TalkWith es una plataforma de comunidades organizada en servidores y canales de texto y de voz. Sus canales de voz agregan subtítulos en vivo, una transcripción completa que se puede buscar y grabación opcional con el consentimiento de todos los participantes.

Problema. En los canales de voz de plataformas como Discord lo que se habla se pierde: no queda registro, no es accesible para personas con dificultades auditivas y quien se pierde parte de la conversación no puede recuperarla.

Solución. El navegador de cada usuario reconoce su voz y la muestra como subtítulo sobre su avatar. Ese texto se guarda como transcripción del canal, que después se consulta y se busca junto con los mensajes de texto. Cualquier participante puede pedir grabar la llamada; la grabación solo inicia cuando todos aceptan y muestra el indicador «Grabando».

Particularidad técnica. El reconocimiento de voz y la grabación de audio ocurren del lado del cliente, sin un servidor de medios propio, lo que encaja con el eje de la materia.

Stack tecnológico

Capa	Tecnología
Frontend	React + Vite + TypeScript + Tailwind CSS + Zustand + React Router
Chat de texto	Socket.io / Firebase Realtime
Llamadas de voz	WebRTC (P2P) · acceso al micrófono con getUserMedia
Subtítulos en vivo	Web Speech API
Grabación	MediaRecorder API
Backend, datos, autenticación y almacenamiento	Firebase (Firestore, Auth con Google, Storage). En esta entrega: API simulada en la capa de servicios (sección 3)
Despliegue	Vercel (frontend) + Render (servidor de señalización)
Versionado	Git + GitHub

Cambios respecto a la documentación inicial: el frontend se implementa en TypeScript, y el acceso es con cuenta de Google más una pantalla de registro para completar el perfil la primera vez.

Cambios en esta entrega

La Entrega 3 conecta las vistas con una capa de datos que se comporta como una API. Se agregaron:

- Capa de servicios con respuestas, errores con código y campo, y persistencia de los cambios (sección 3).
- Manejo de errores en un solo lugar y estados de carga, error y vacío en las vistas que consumen datos.
- **CRUD completo de canales**: listar, ver detalle, crear, editar y eliminar con confirmación (sección 4.8).
- Formularios validados: inicio de sesión con correo, registro, crear servidor, unirse con invitación y crear o editar canal.
- Crear servidores y unirse con un enlace de invitación real.
- Acceso real al micrófono en los canales de voz, con avisos de permisos y ajustes de audio.
- Pantallas 12 a 16 del canal de voz y grabación, que en la entrega anterior estaban pendientes.

2. Arquitectura del frontend

Las vistas están hechas en React con TypeScript. Ningún componente lee los datos directamente: todo pasa por los servicios de `src/services`, que en esta entrega simulan una API (sección 3). Los estilos usan los tokens del sistema de diseño de Figma (colores, tipografía Inter y sombras) definidos como clases de Tailwind en `src/index.css`.

La entrega usa términos de Angular. La siguiente tabla muestra cómo se cubre cada uno en React:

Elemento solicitado	Implementación en React	Ejemplos en el proyecto
Componentes	Componentes `.tsx` reutilizables y páginas	Button, Field, TextArea, Modal, Avatar, Badge, Switch, Dropdown, ServerBar, ChannelSidebar
Directivas	Custom hooks de comportamiento y el componente `<Can>`, que muestra u oculta contenido según el rol	useAutoScroll, useClickOutside, useDebounce, useHotkey, useService, useMicrofono, <Can roles=[…]>
Servicios	Módulos en `src/services` que responden como una API: devuelven una Promise, validan los datos y lanzan `ApiError` cuando algo falla	serverService, channelService, inviteService, authService, sessionService, searchService
Manejo de errores	`src/services/errores.ts`: `ApiError`, `mensajeDeError()` y `manejarError()`	manejarError(error, 'No se pudo crear el canal')
Validaciones	Reglas en `src/lib/validaciones.ts`, compartidas por formularios y servicios	validarCanal, validarServidor, validarLogin, validarRegistro
Guardias / middlewares	Componentes que envuelven grupos de rutas y redirigen si no se cumple la condición	ProtectedRoute, GuestRoute, MemberGuard, RoleGuard
Rutas	React Router con rutas anidadas y parámetros	src/routes/AppRouter.tsx
Estado global	Zustand: sesión simulada, notificaciones, confirmaciones y estado de demo de voz	authStore, uiStore, voiceStore
Estado de cada vista	Hook `useService`: datos, carga, error y recargar	const { data, loading, error, recargar } = useService(…)

Rutas de la aplicación

Ruta	Pantallas	Guardia
`/`	01 Landing	—
`/login`, `/registro`	02–04 Autenticación	GuestRoute
`/app`	05 Inicio	ProtectedRoute
`/app/servidores`, `/nuevo`, `/app/unirse`	06–09 Servidores	ProtectedRoute
`/app/s/:serverId/c/:channelId`	10–11 Texto · 12–16 Voz	MemberGuard
`/app/s/:serverId/sesiones/:sessionId`	17–20 Historial y sesión	MemberGuard
`/app/s/:serverId/grabaciones`	21 Grabaciones	MemberGuard
`/app/s/:serverId/ajustes/…`	24–27 Configuración	RoleGuard
`/app/buscar`	22–23 Búsqueda	ProtectedRoute
`/app/perfil`	28 Perfil	ProtectedRoute
cualquier otra	29 No encontrada	—

Para la demostración, un panel de «Modo demo» permite simular el rol de miembro: así se comprueba que RoleGuard bloquea la configuración del servidor y muestra el mensaje de acceso denegado (403).

3. Datos, formularios y manejo de errores

3.1 Fuente de datos: API simulada en la capa de servicios

Para esta entrega elegimos una **API simulada dentro de la capa de servicios**. Cada función de `src/services/index.ts` se comporta como un endpoint: responde después de un pequeño retardo, valida los datos igual que lo haría el backend y, si algo falla, rechaza la promesa con un `ApiError` que trae el código HTTP equivalente (400, 401, 404 o 409) y, cuando aplica, el campo que falló. Los cambios en canales, servidores, miembros e invitaciones se guardan en `localStorage`, así que se conservan al recargar la página.

Por qué. El backend definitivo del proyecto es Firebase (Firestore y Auth con Google) más un servidor Node.js para Socket.io y la señalización de WebRTC. Esas tareas siguen abiertas en el cronograma. Simular la API en los servicios nos permitió terminar y probar los formularios, el CRUD y los estados de la interfaz sin depender del backend.

Cómo se reemplazará. Las vistas solo conocen las funciones de los servicios y el formato de sus errores. Al conectar Firebase se cambia el cuerpo de cada función por la llamada real y se lanza el mismo `ApiError` cuando el servidor responda con error. Los componentes, los formularios y el manejo de errores no cambian. Las llaves de Firebase se leerán de variables de entorno de Vite (`import.meta.env`), lo cual queda pendiente para la siguiente entrega.

3.2 Capa de servicios y manejo de errores

- Todas las operaciones de datos viven en `src/services`. Los componentes nunca importan los datos de ejemplo de `src/mocks`.
- `ApiError(status, mensaje, campo)` representa un error de la API. El campo indica qué input del formulario falló.
- `mensajeDeError(error)` convierte cualquier error en un texto para la persona, y `manejarError(error, titulo)` lo muestra como notificación de error. Es el único lugar donde se muestran los errores de la API.
- Si el error trae un campo (por ejemplo, un nombre repetido), el formulario lo muestra debajo de ese campo en lugar de una notificación.
- El hook `useService(fn, deps)` llama al servicio y devuelve `data`, `loading`, `error` y `recargar`, con lo que cada vista pinta sus estados de carga, error y vacío.

3.3 Contrato de la API

Cada operación equivale a un endpoint. La columna «Errores» indica el código y el mensaje que devuelve el servicio.

Operación	Función del servicio	Respuesta	Errores
Listar canales	`channelService.getByServer(servidorId)`	Canal[]	—
Ver canal	`channelService.getById(canalId)`	Canal	404 El canal no existe o fue eliminado
Crear canal	`channelService.create(servidorId, datos)`	Canal	400 regla de validación · 409 nombre repetido
Editar canal	`channelService.update(canalId, datos)`	Canal	400 · 404 · 409
Eliminar canal	`channelService.remove(canalId)`	—	404 ya había sido eliminado
Listar mis servidores	`serverService.getMine()`	Servidor[]	—
Crear servidor	`serverService.create(datos)`	{ servidor, invitacion }	400 · 409 ya perteneces a uno con ese nombre
Validar invitación	`inviteService.validate(enlace)`	Invitacion o null	—
Unirse a un servidor	`inviteService.join(enlace)`	Servidor	404 enlace no válido · 409 ya eres miembro
Iniciar sesión	`authService.login(correo, contrasena)`	Usuario	401 Correo o contraseña incorrectos
Registro	`authService.registrar({ nombre, usuario })`	Usuario	409 Ese nombre de usuario ya está en uso

Modelos de datos (definidos en `src/types/index.ts`):

Modelo	Campos
Canal	canalId, servidorId, nombre, tipo ('texto' | 'voz'), descripcion?, privado?
Servidor	servidorId, nombre, descripcion, iniciales, color, propietarioUid, privacidad ('invitacion' | 'enlace'), totalMiembros
Miembro	miembroId, servidorId, uid, rol ('propietario' | 'admin' | 'miembro'), unidoEn
Invitacion	codigo, servidorId, invitaUid, expiraEn, enLinea, enLlamada

Modelo	Campos
Usuario	uid, nombre, usuario, correo, iniciales, color, estado
ApiError	status (400, 401, 404, 409), message, campo?

3.4 Reglas de validación

Las reglas están en `src/lib/validaciones.ts`. Los formularios las usan para mostrar errores mientras se escribe, y los servicios las vuelven a revisar antes de guardar, igual que haría el backend.

Formulario	Campo	Regla
Canal	Nombre	Obligatorio, de 2 a 30 caracteres. En canales de texto solo minúsculas, números y guiones, sin guion al inicio o al final. No se puede repetir en el servidor.
Canal	Descripción	Máximo 120 caracteres.
Canal	Roles	Si el acceso es privado, al menos un rol autorizado.
Servidor	Nombre	Obligatorio, de 3 a 50 caracteres. No se puede repetir entre tus servidores.
Servidor	Descripción e imagen	Descripción de máximo 200 caracteres; imagen PNG o JPG de máximo 5 MB.
Inicio de sesión	Correo	Obligatorio y con formato válido (nombre@dominio.com).
Inicio de sesión	Contraseña	Obligatoria, mínimo 8 caracteres.
Registro	Nombre visible	De 2 a 32 caracteres.
Registro	Nombre de usuario	De 3 a 20 caracteres; solo minúsculas, números y puntos; único.
Registro	Casillas	Aceptar los términos y la política de grabación.
Unirse	Enlace o código	Debe corresponder a una invitación vigente.

En todos los formularios los errores aparecen debajo de cada campo cuando la persona sale del campo o intenta enviar. Mientras se envía, el botón se desactiva y cambia a «Guardando…», «Creando…», «Verificando…» o «Uniéndote…». Al terminar se muestra una notificación de éxito, o el error en el campo o como notificación.

3.5 Estados de la interfaz

Toda vista que consume un servicio maneja tres estados además del normal:

- **Carga:** un indicador como «Cargando canales…», tarjetas grises o «Cargando tu resumen…».
- **Error:** un recuadro con el mensaje del servicio y el botón «Reintentar», que vuelve a pedir los datos con `recargar()`.
- **Vacío:** un mensaje con la acción para crear el primer elemento, por ejemplo «Este servidor todavía no tiene canales».



Figura 1. *Estado de carga del Inicio mientras responde el servicio.*

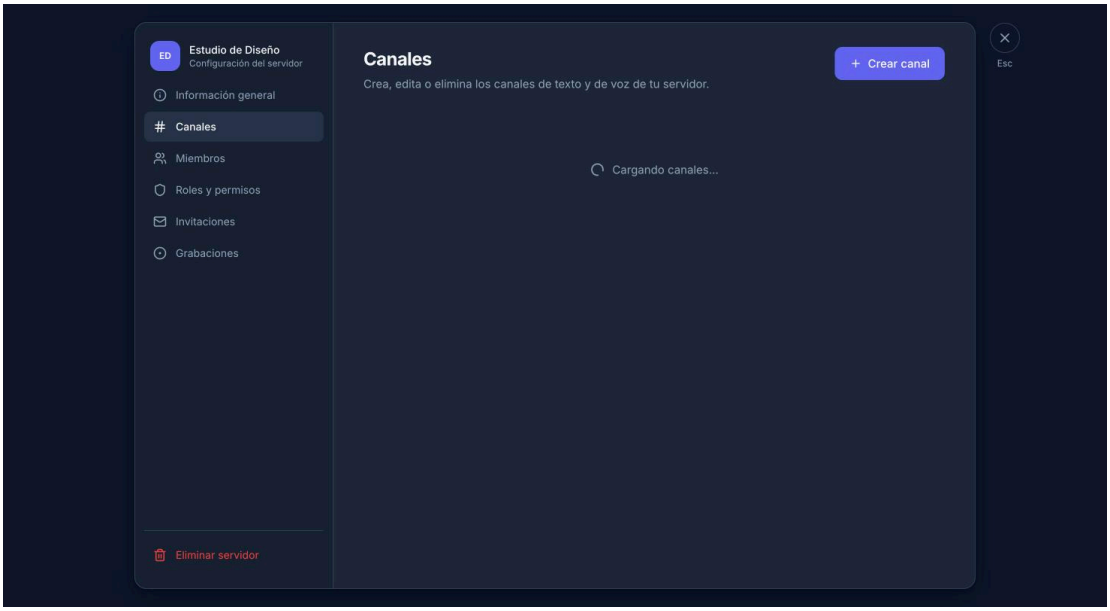


Figura 2. *Estado de carga en la lista de canales.*

4. Secciones de la aplicación

Las pantallas siguen la numeración del diseño en Figma y se agrupan por flujo. Las pantallas internas comparten el mismo layout: barra de servidores a la izquierda, columna de canales del servidor activo con los participantes de cada canal de voz, panel del usuario actual y barra superior con el buscador global (Ctrl K). Las capturas marcadas como **Nuevo** corresponden a esta entrega.

4.1 Página pública

Son las vistas que cualquier persona puede abrir sin iniciar sesión.

01 Landing /

Presenta TalkWith: navegación con «Iniciar sesión» y «Crear cuenta», el lema «Lo que se dice en voz, ya no se pierde», una vista previa de una llamada con subtítulos y transcripción en vivo, el problema que resuelve, sus funciones, los pasos para empezar y un llamado a la acción.

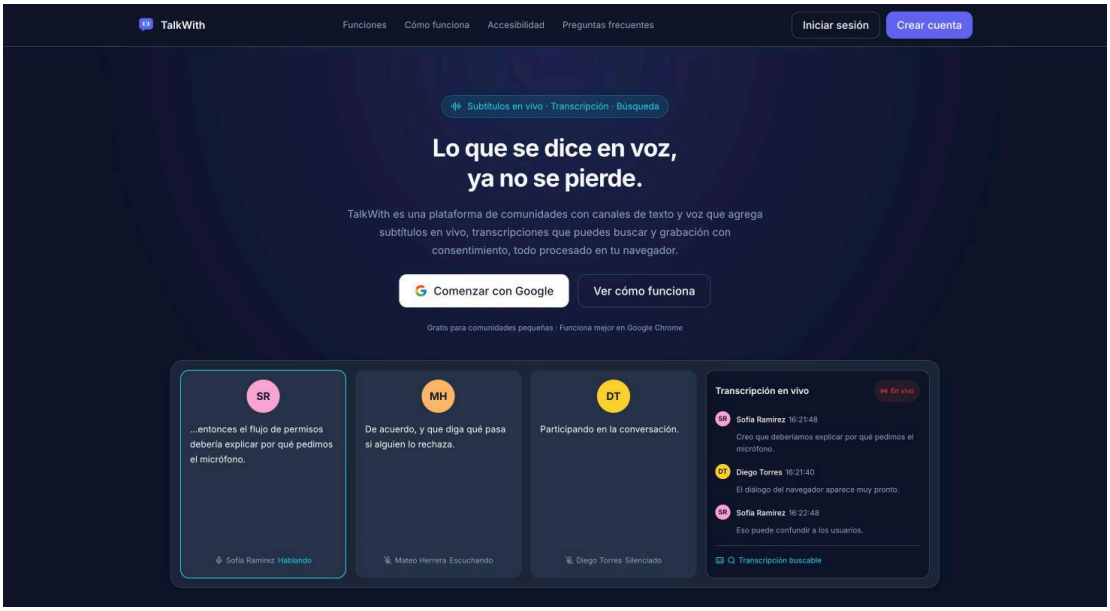


Figura 3. *Landing de TalkWith.*

29 Página no encontrada *

Aparece cuando la ruta no existe, el canal ya no está o la persona no es miembro del servidor. Ofrece volver al inicio o ver sus servidores. La misma vista muestra un «403 · No tienes permiso» cuando la guardia de rol bloquea el acceso (sección 4.8).

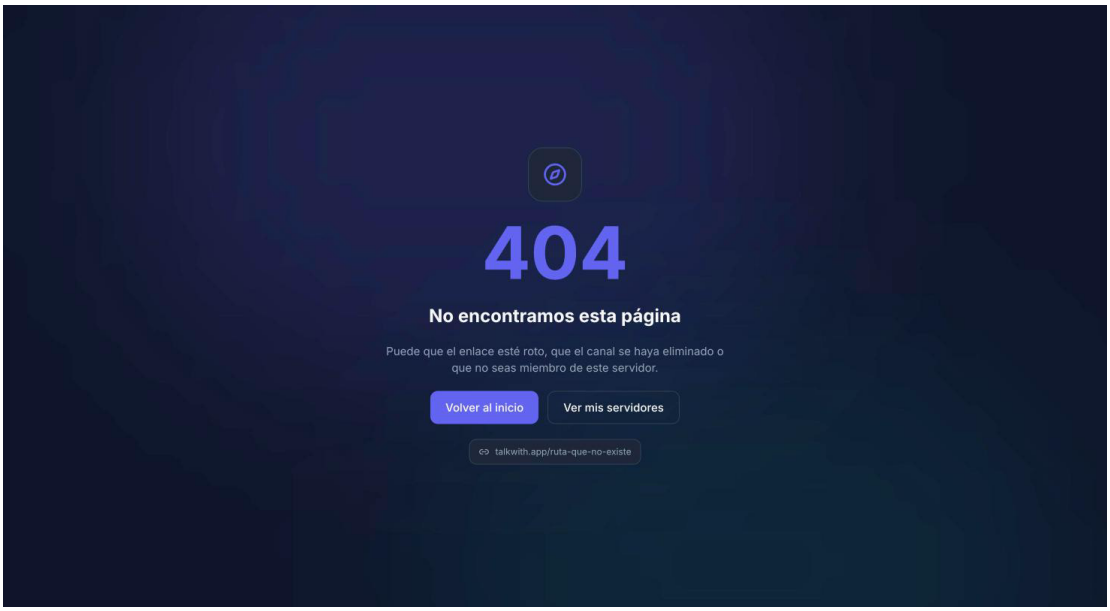


Figura 4. *Página no encontrada.*

4.2 Pantallas de autenticación de usuario

El acceso principal es con la cuenta de Google, que en esta entrega sigue simulado. Se agregó además un formulario de correo y contraseña con validación, que envía los datos a `authService`; todavía no se conecta con el manejo de sesión por token. Ambas rutas usan GuestRoute: si ya hay una sesión activa, redirigen al inicio.

02 Inicio de sesión `/login`

Tarjeta de bienvenida con el botón «Continuar con Google» y, debajo, el formulario de correo y contraseña. Si la persona intentó abrir una página protegida, al entrar regresa a esa página.

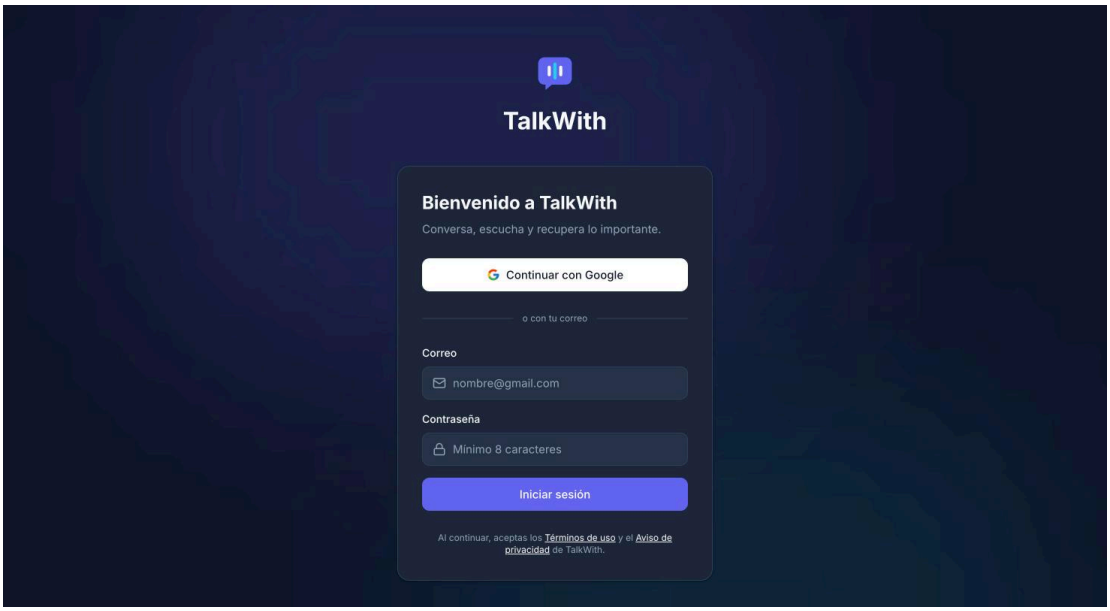


Figura 5. *Inicio de sesión con Google o con correo.*

03 Inicio de sesión — cargando

Mientras se valida la cuenta, el botón se desactiva y muestra «Iniciando sesión» con un indicador de carga. Los campos del formulario también se desactivan.

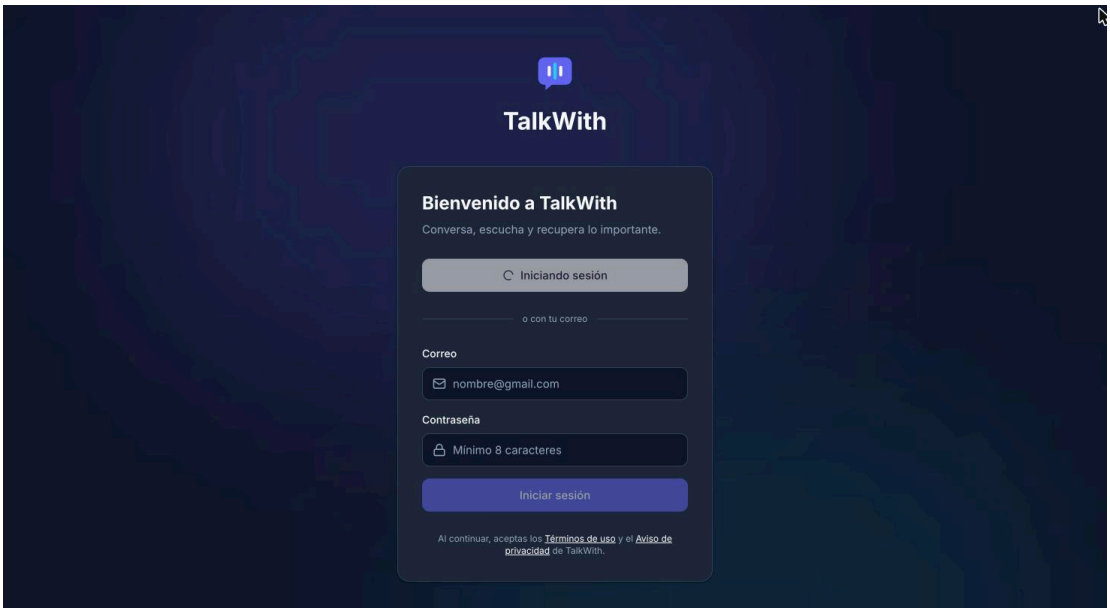


Figura 6. *Inicio de sesión en estado de carga.*

Nuevo Validación del inicio de sesión

El correo debe tener un formato válido y la contraseña al menos 8 caracteres; cada error aparece debajo de su campo. Si los datos son correctos pero no coinciden con una cuenta, el servicio responde 401 y se

muestra «Correo o contraseña incorrectos» arriba del formulario. Mientras se verifica, el botón cambia a «Verificando…».

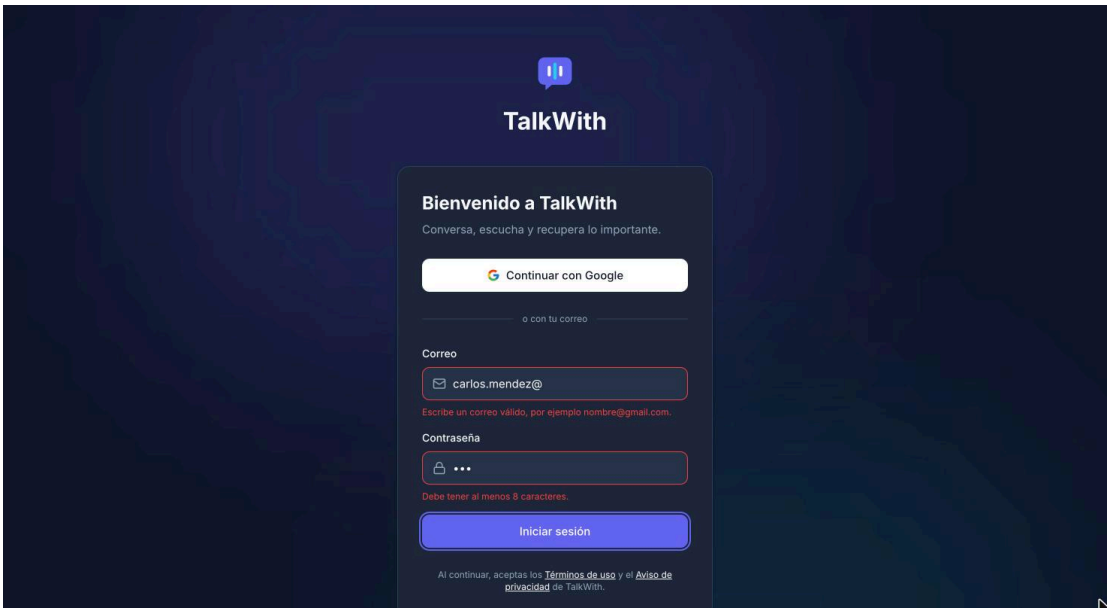


Figura 7. *Errores por campo en el inicio de sesión.*

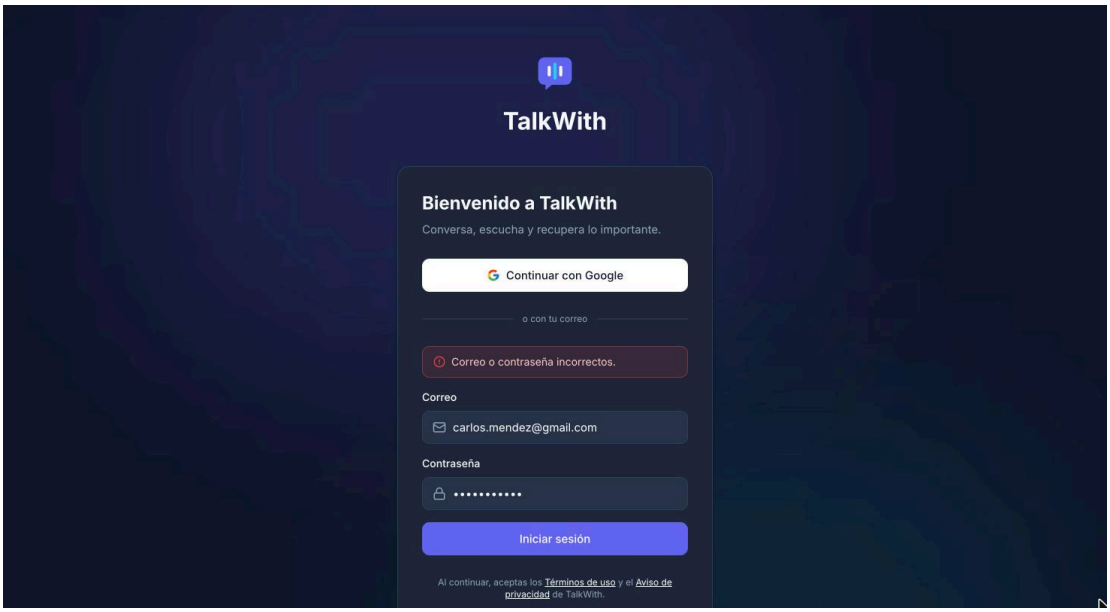


Figura 8. *Respuesta de error del servicio: credenciales incorrectas.*

04 Registro — completar perfil `/registro`

Se muestra la primera vez que alguien entra con Google. Usa la foto de la cuenta, pide un nombre visible y un nombre de usuario único, y muestra el correo bloqueado. Las reglas son las de la sección 3.4. Al enviar, `authService.registrar` revisa que el nombre de usuario no esté ocupado; si lo está, el error aparece en ese campo.

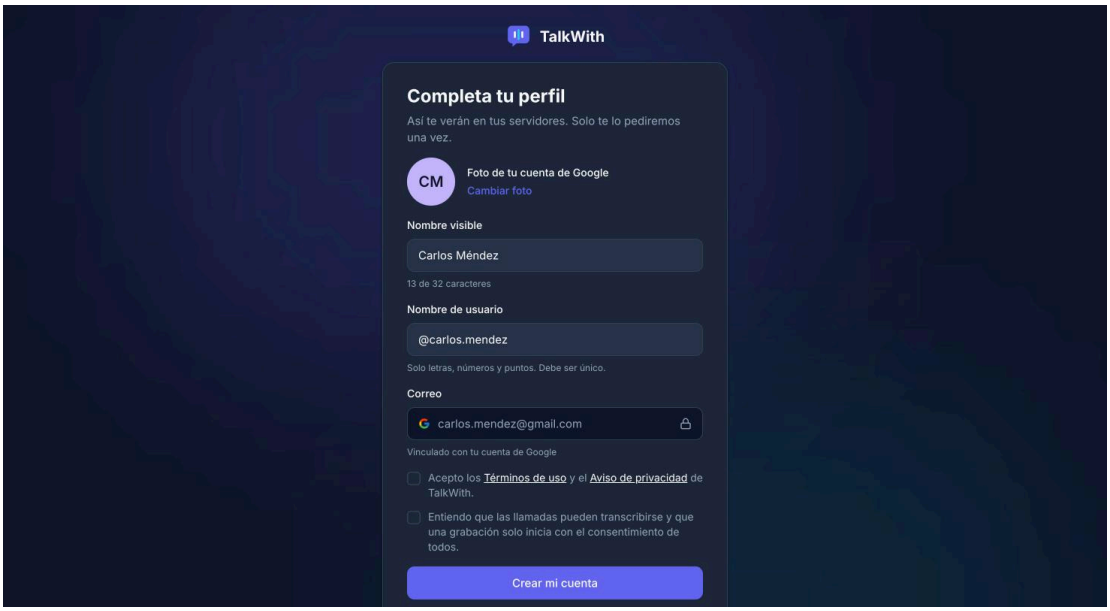


Figura 9. *Registro: completar perfil.*

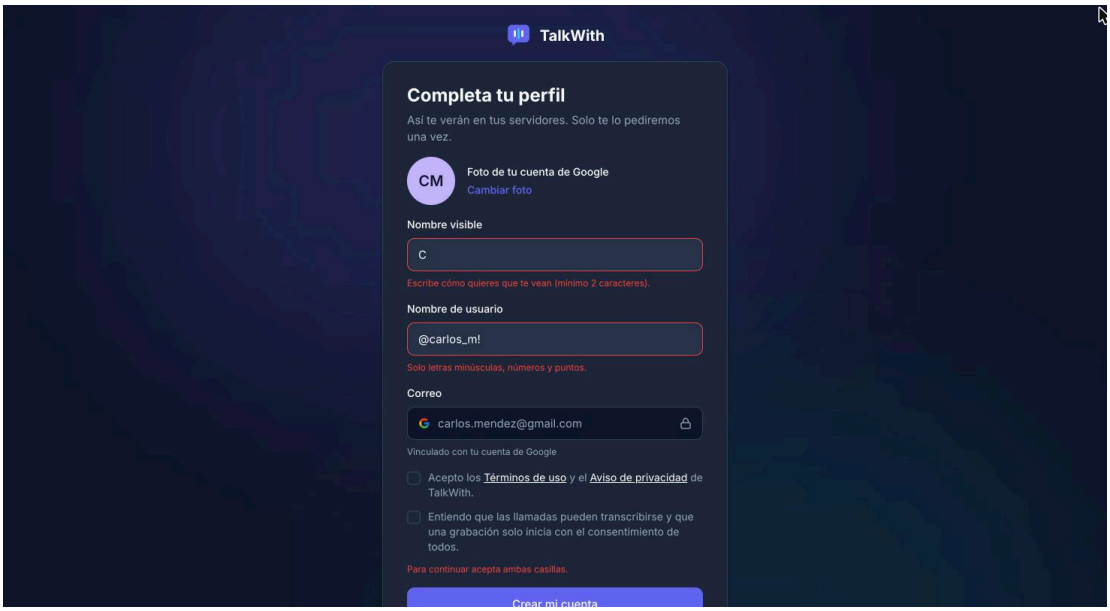


Figura 10. *Nuevo: errores de validación del registro.*

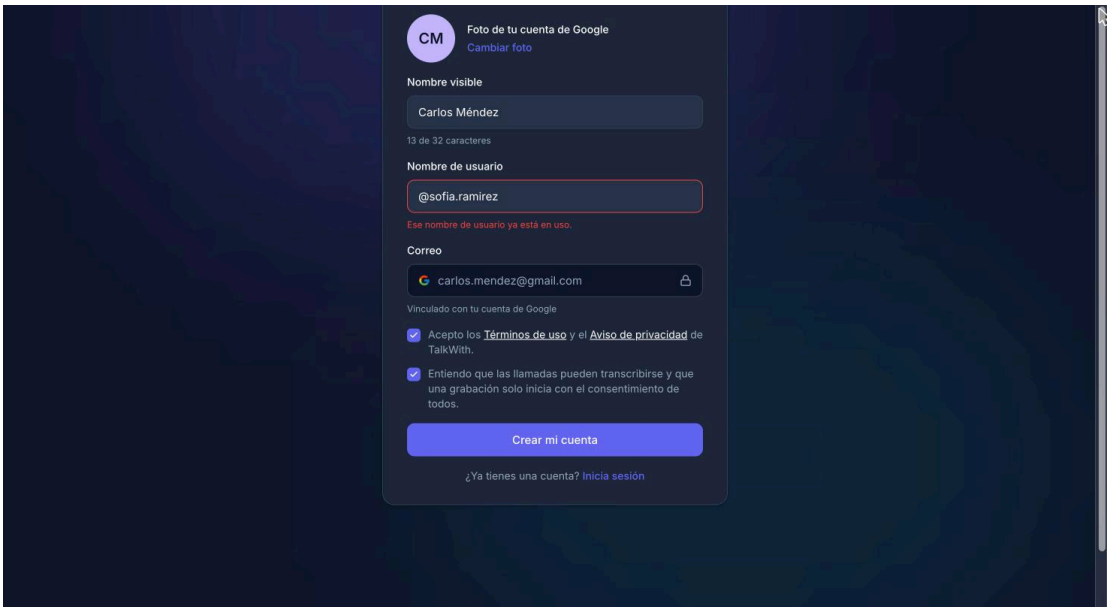


Figura 11. *Nuevo: el servicio indica que el nombre de usuario ya está en uso.*

4.3 Inicio y servidores

05 Inicio (dashboard) /app

Saludo según la hora y resumen de actividad. Una tarjeta invita a continuar en la llamada activa de Sala principal, con los participantes y el último subtítulo en vivo. Debajo hay accesos rápidos a transcripciones recientes, la búsqueda y crear un servidor, y la lista de actividad reciente.



Figura 12. *Inicio del portal.*

06 Tus servidores `/app/servidores`

Listado en tarjetas de los servidores del usuario con su rol, descripción, número de miembros y canales, y cuántas personas están en llamada. Se puede filtrar por rol. La lista se pide a `serverService.getMine()` y maneja los estados de carga, error con «Reintentar» y vacío. Los servidores que se crean o a los que la persona se une aparecen aquí y en la barra lateral.

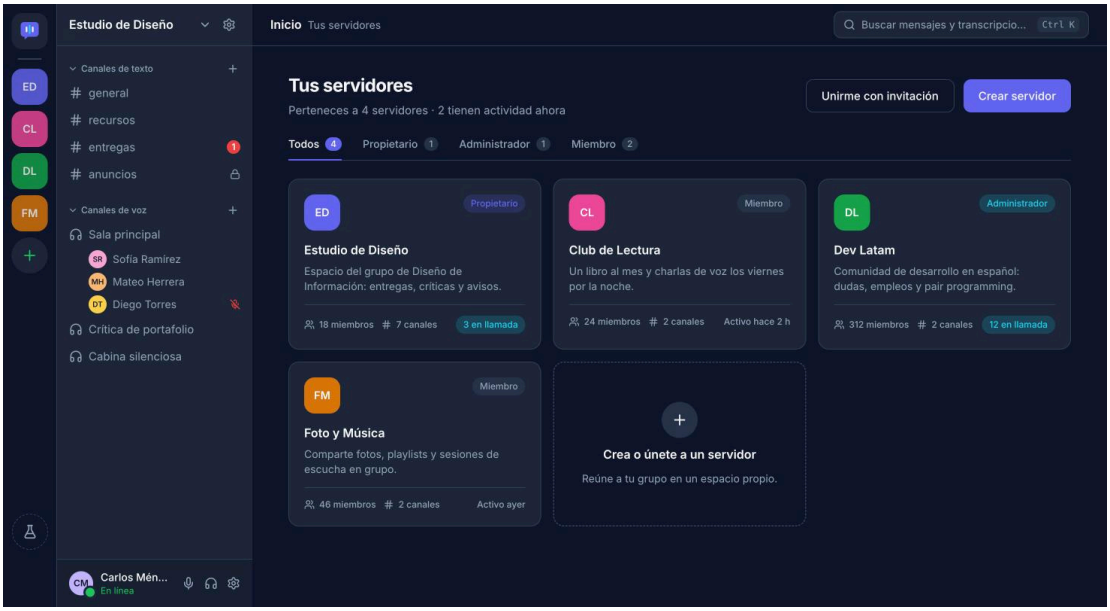


Figura 13. *Listado de servidores.*

07 Crear servidor `/app/servidores/nuevo`

Formulario en modal con imagen opcional, nombre con contador de caracteres, descripción y privacidad (solo por invitación o abierto con enlace). Ahora valida el nombre y la descripción, y el servicio rechaza un nombre que ya tenga otro de tus servidores.

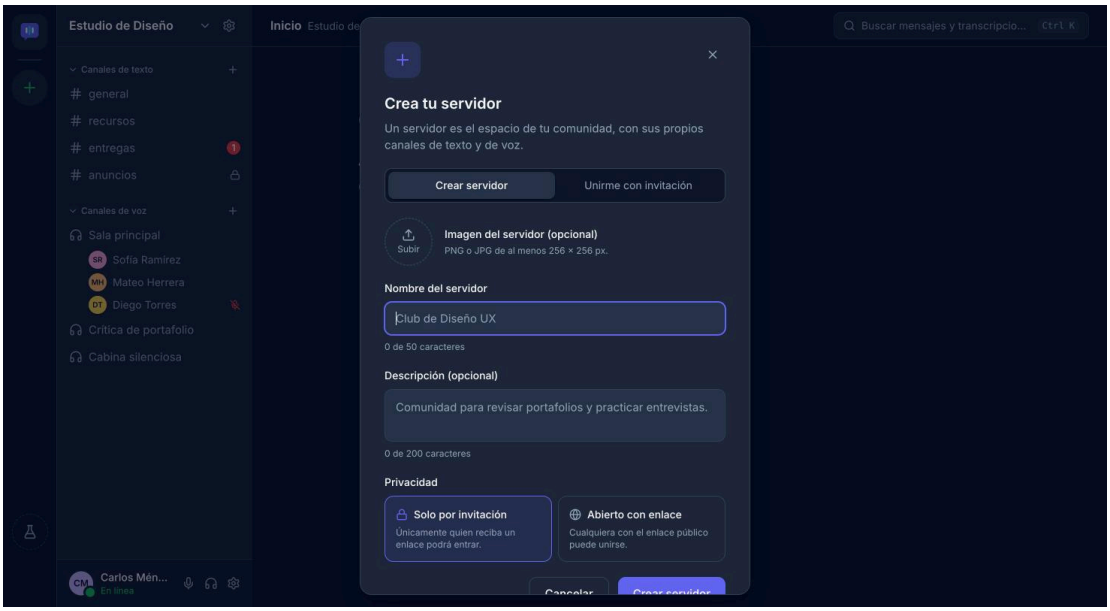


Figura 14. *Formulario para crear un servidor.*

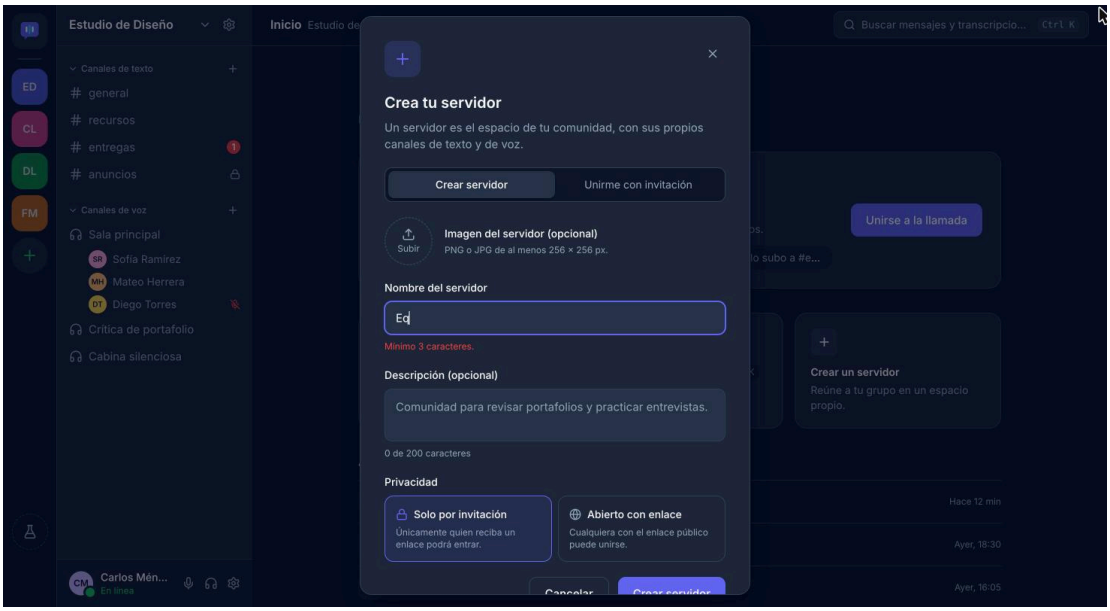


Figura 15. *Nuevo: error de validación del nombre.*

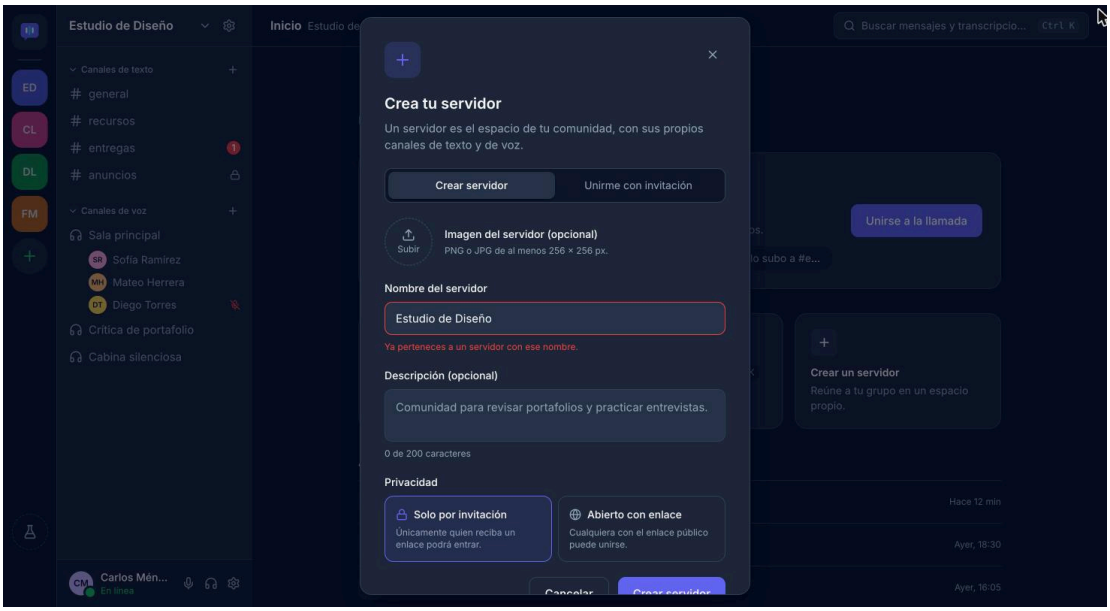


Figura 16. *Nuevo: error devuelto por el servicio, nombre repetido.*

09 Servidor creado y notificaciones

Al crear, el servicio guarda el servidor con un canal de texto #general y un canal de voz, registra a la persona como propietaria y genera un enlace de invitación que se copia al portapapeles. La aplicación abre el nuevo servidor y muestra las notificaciones de éxito y del enlace copiado.

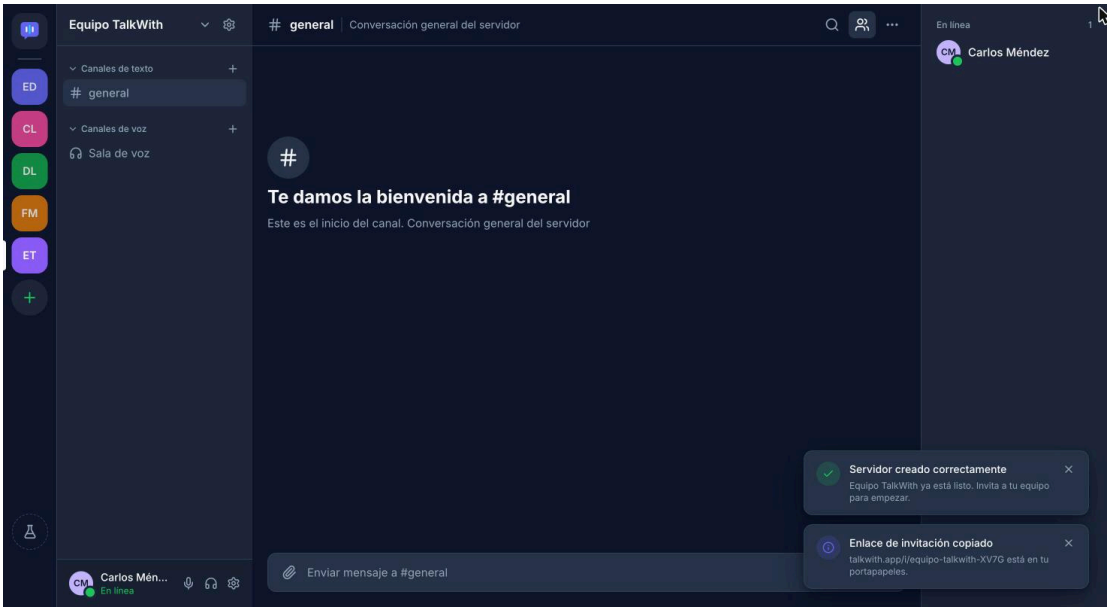


Figura 17. *Servidor recién creado con sus notificaciones.*

08 Unirse con invitación `/app/unirse`

Campo para pegar el enlace o código de invitación. La invitación se valida mientras se escribe (con debounce): si no existe, el error aparece en el campo; si es válida, se muestra una vista previa del

servidor. Al pulsar «Unirme al servidor», `inviteService.join` agrega a la persona como miembro y la lleva al servidor. Si ya era miembro, el botón cambia a «Ir al servidor».

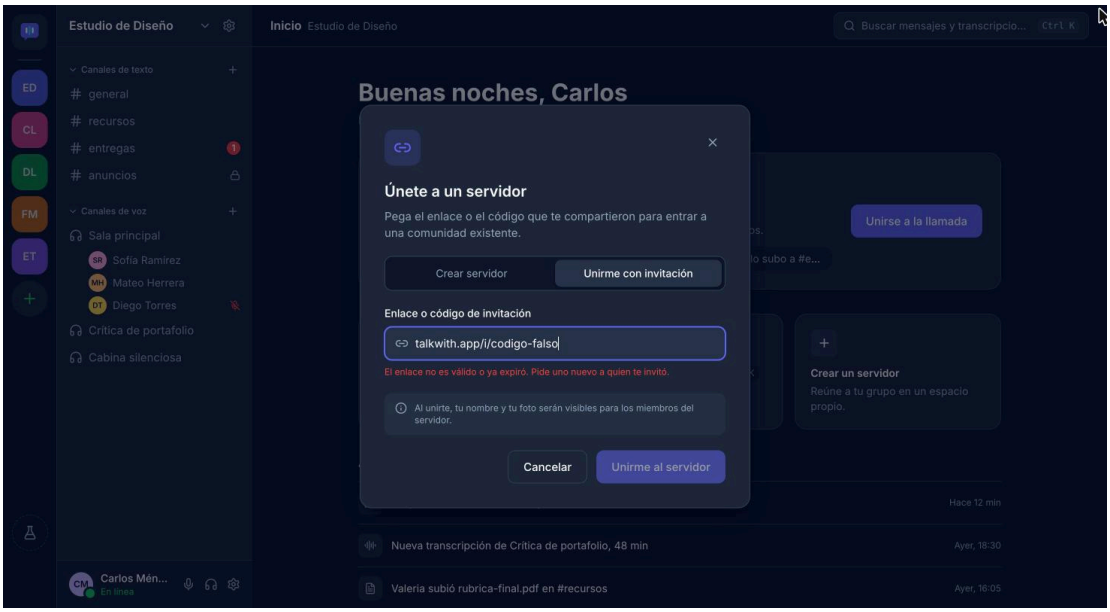


Figura 18. *Nuevo: invitación no válida.*

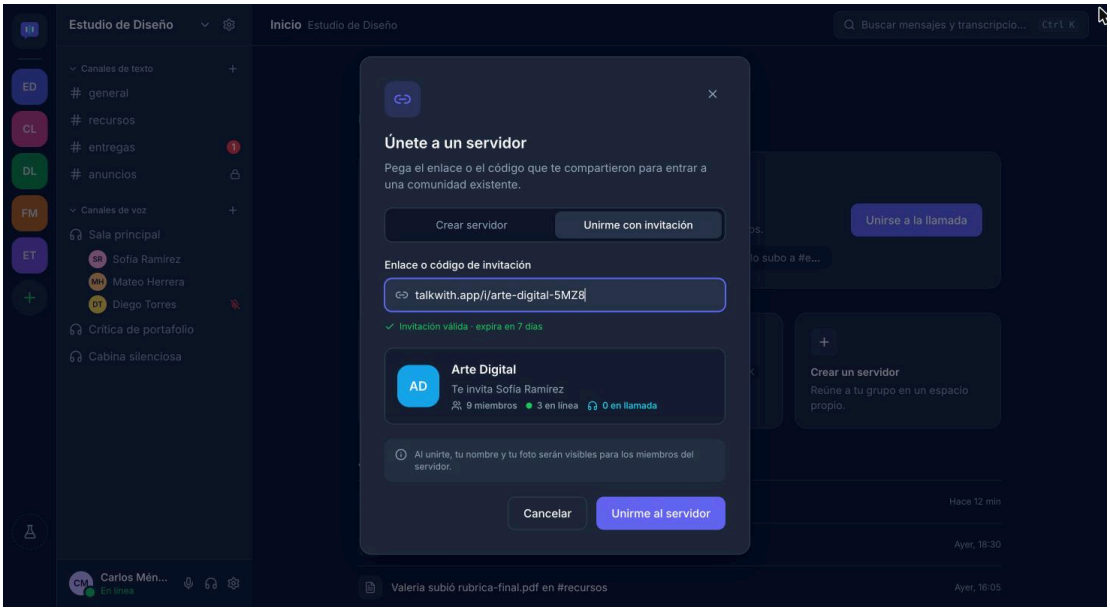


Figura 19. *Invitación válida con la vista previa del servidor.*

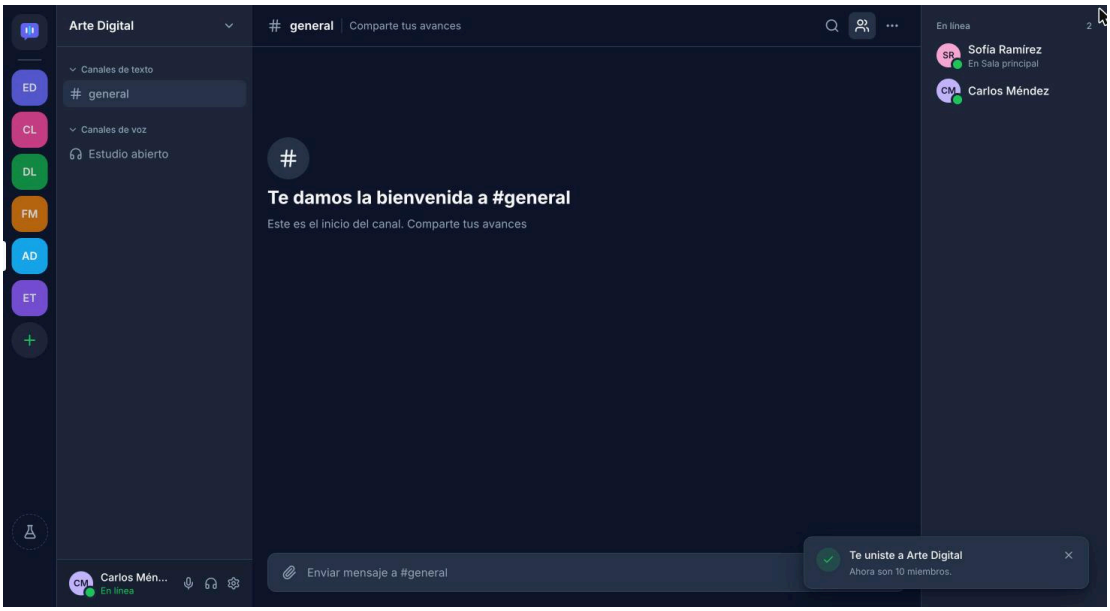


Figura 20. *Nuevo: la persona se unió al servidor y se abre su canal #general.*

4.4 Canal de texto

10 Canal de texto `/app/s/:serverId/c/:channelId`

Historial de mensajes agrupados por autor y fecha, con archivos adjuntos, vistas previas de enlaces y menciones resaltadas. Incluye el indicador «Isaian está escribiendo…», la caja para enviar mensajes y la columna de miembros agrupada en Coordinación, en línea y desconectados. Solo pueden abrirlo los miembros del servidor (MemberGuard).

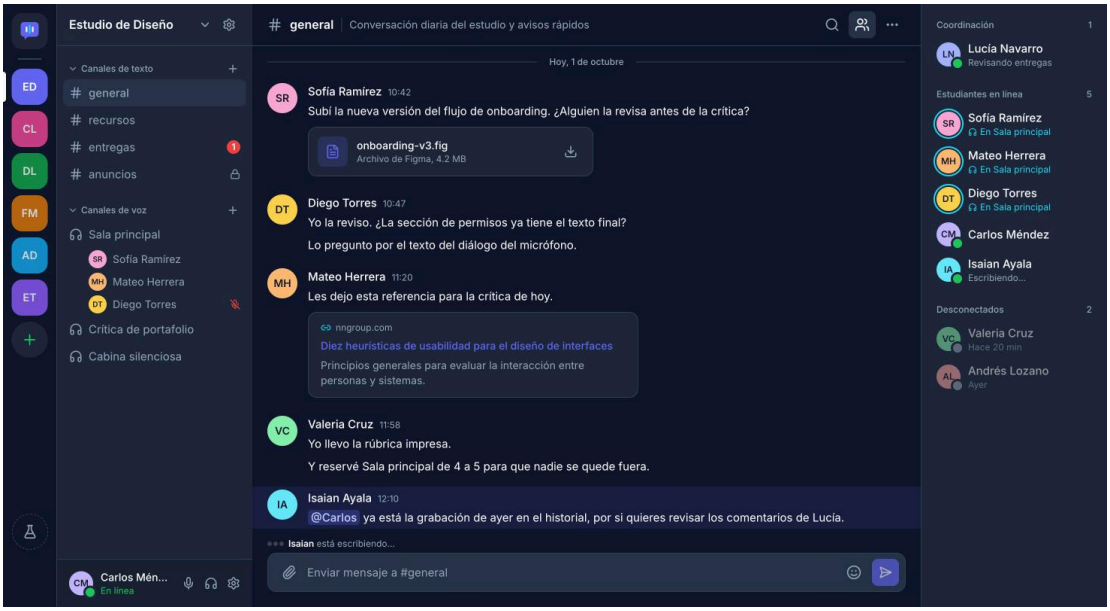


Figura 21. *Canal de texto #general.*

11 Canal de texto — reconectando

Desde el menú del canal se simula una pérdida de conexión: aparece un aviso con «Reintentar ahora» y la caja de mensaje indica que lo escrito se enviará al reconectar.

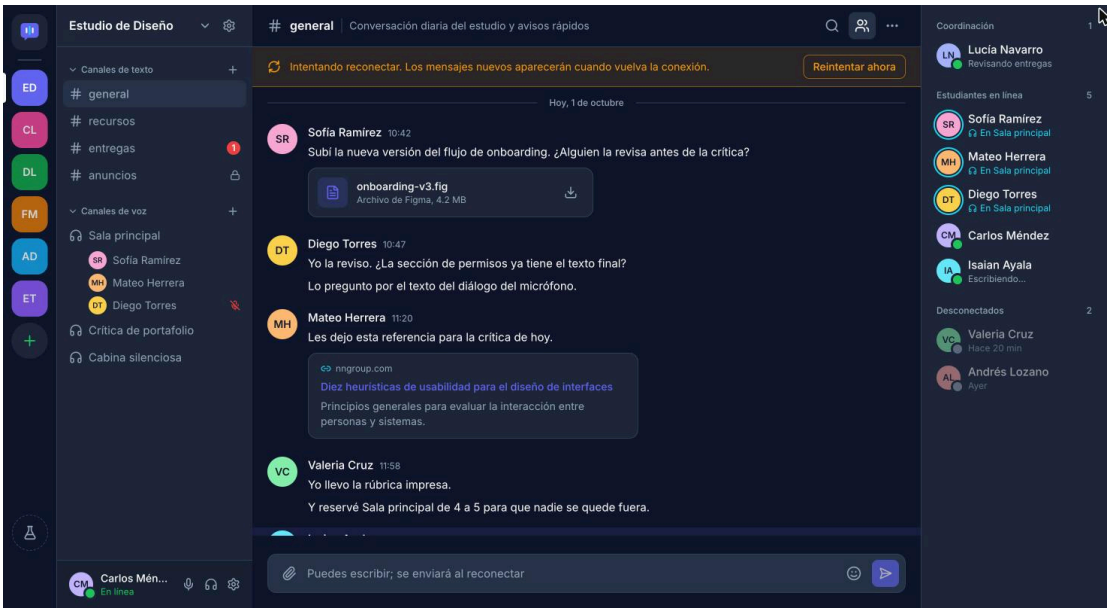


Figura 22. *Canal de texto intentando reconectar.*

4.5 Canal de voz y grabación

Es la vista central de TalkWith y se abre en la misma ruta que los canales de texto cuando el canal es de voz. Estas pantallas estaban pendientes en la entrega anterior.

12 Canal de voz con subtítulos

Cuadrícula de participantes con su avatar, el subtítulo en vivo sobre quien habla y su estado (hablando, silenciado o seleccionado). Un panel lateral muestra la transcripción en vivo con autor y hora. Los controles permiten silenciar el micrófono, abrir los ajustes de audio, activar los subtítulos, ver participantes, solicitar grabación y salir. **Nuevo:** al entrar se pide permiso real para usar el micrófono; mientras el navegador pregunta, un aviso lo explica y la persona entra silenciada.

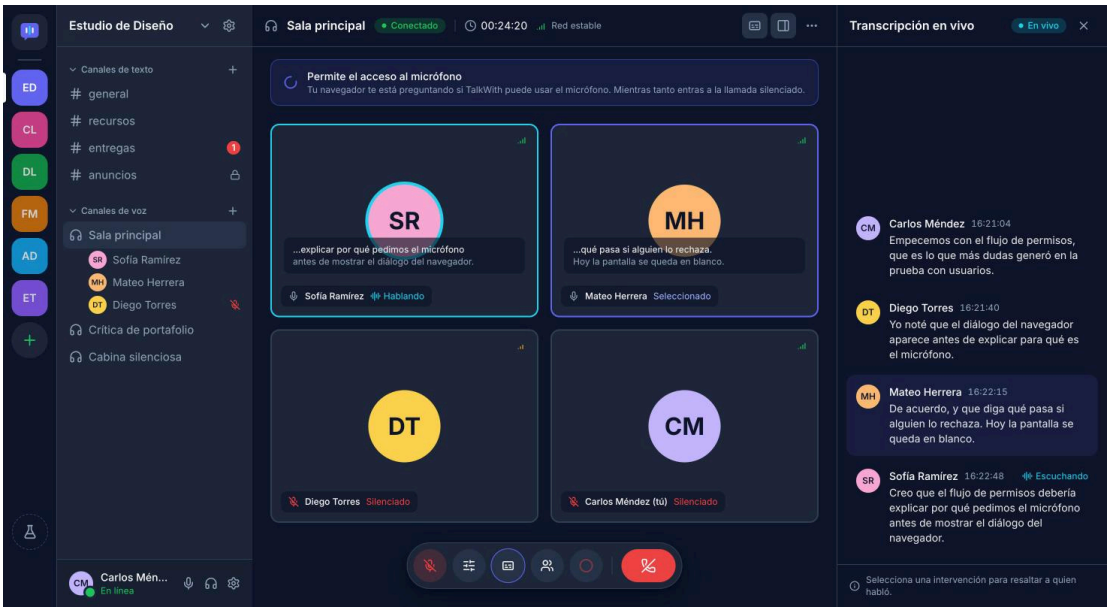


Figura 23. *Canal de voz con subtítulos y aviso de permiso del micrófono.*

13 Micrófono bloqueado y subtítulos no disponibles

Si el permiso del micrófono está bloqueado, el aviso superior explica cómo activarlo y ofrece «Reintentar». Como los subtítulos dependen del micrófono, el panel de transcripción indica que no se pueden generar, ofrece revisar los permisos e indica que funciona en Chrome y Edge. También se muestran avisos cuando no hay micrófono conectado o el navegador no lo permite.

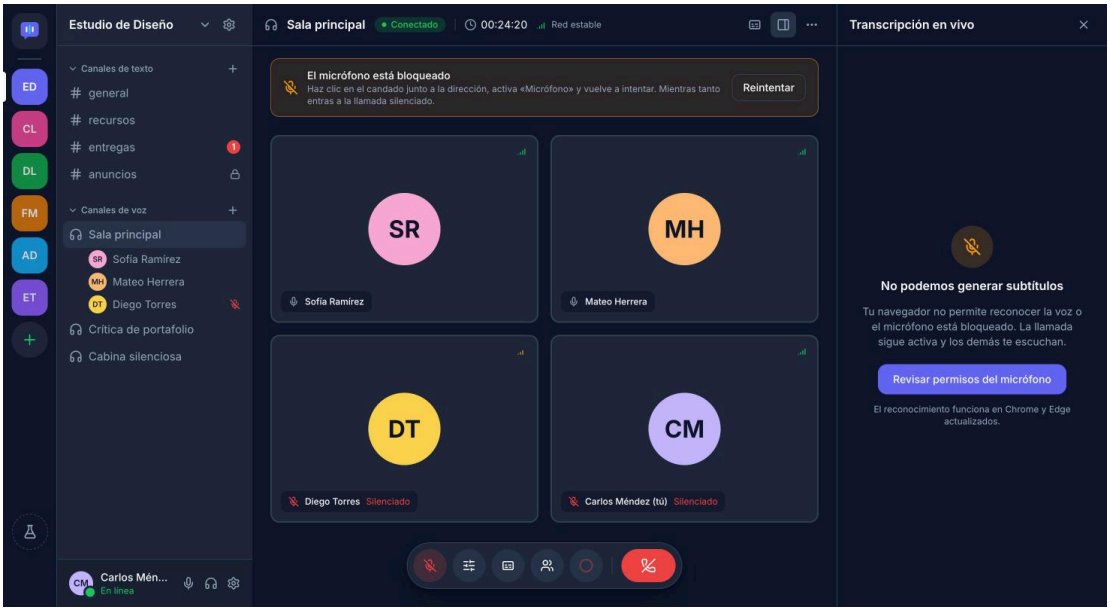


Figura 24. *Micrófono bloqueado y subtítulos no disponibles.*

14–15 Solicitud y consentimiento de grabación

Quien solicita ve quién ha aceptado, rechazado o sigue pendiente. Los demás reciben un modal para aceptar o rechazar. La grabación solo inicia con el consentimiento de todos.

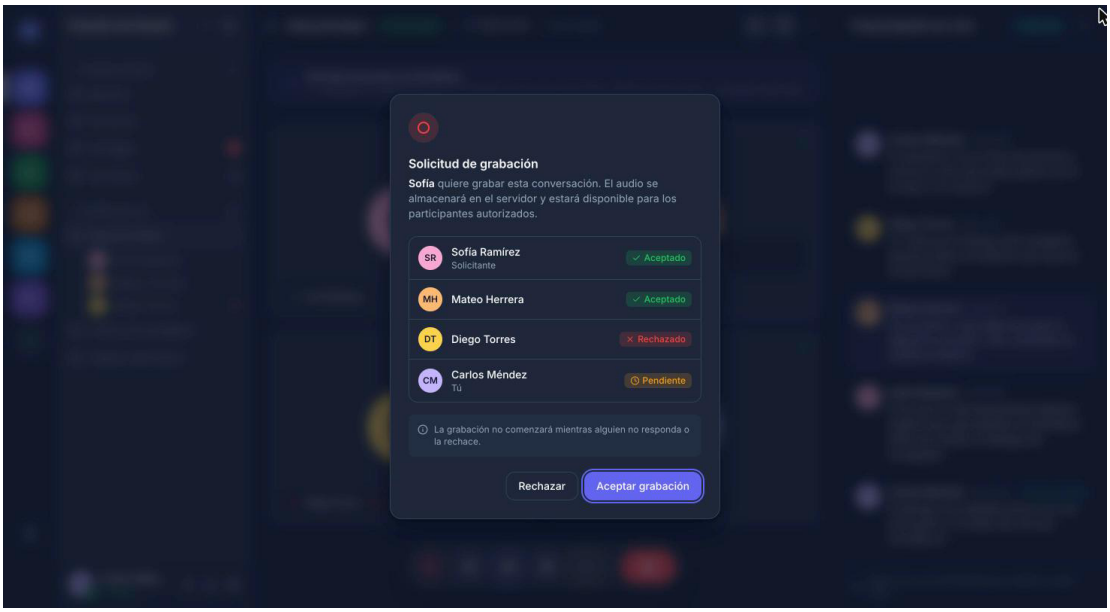


Figura 25. *Solicitud de grabación con el estado de cada participante.*

16 Grabación activa

Una franja roja con «Grabando», el tiempo transcurrido y «Visible para todos los participantes» se mantiene durante toda la llamada. Detener la grabación o salir pide confirmación.

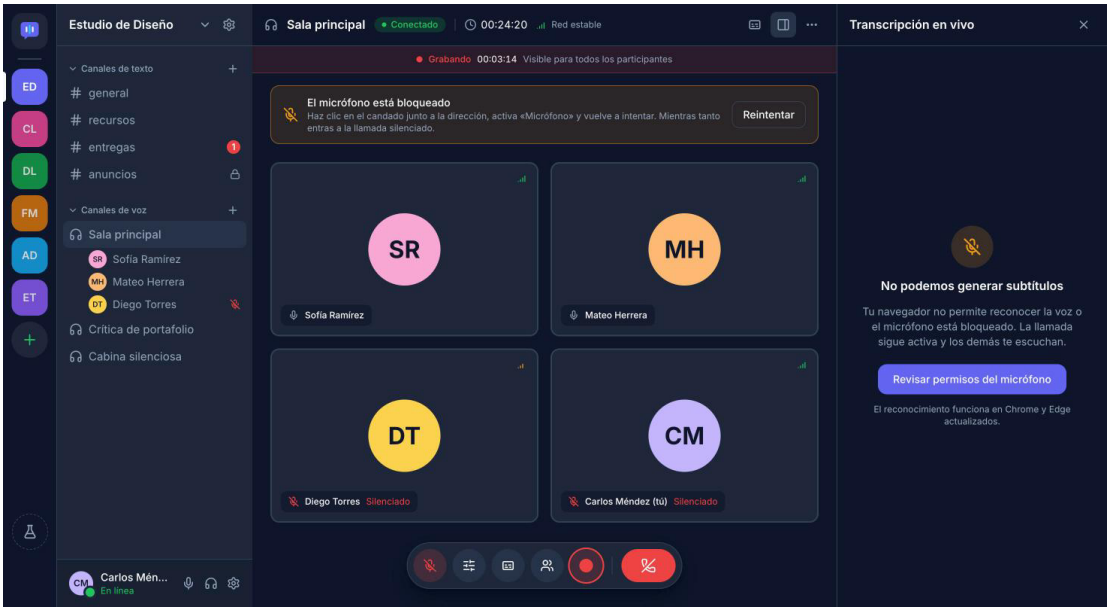


Figura 26. *Grabación activa.*

Nuevo Ajustes de audio

El botón de ajustes abre una ventana para elegir el micrófono y un medidor de nivel para probarlo. Si no hay permiso, explica el problema y ofrece «Reintentar». Silenciar apaga la pista de audio real, y la tarjeta propia muestra «Hablando» cuando el micrófono detecta voz.

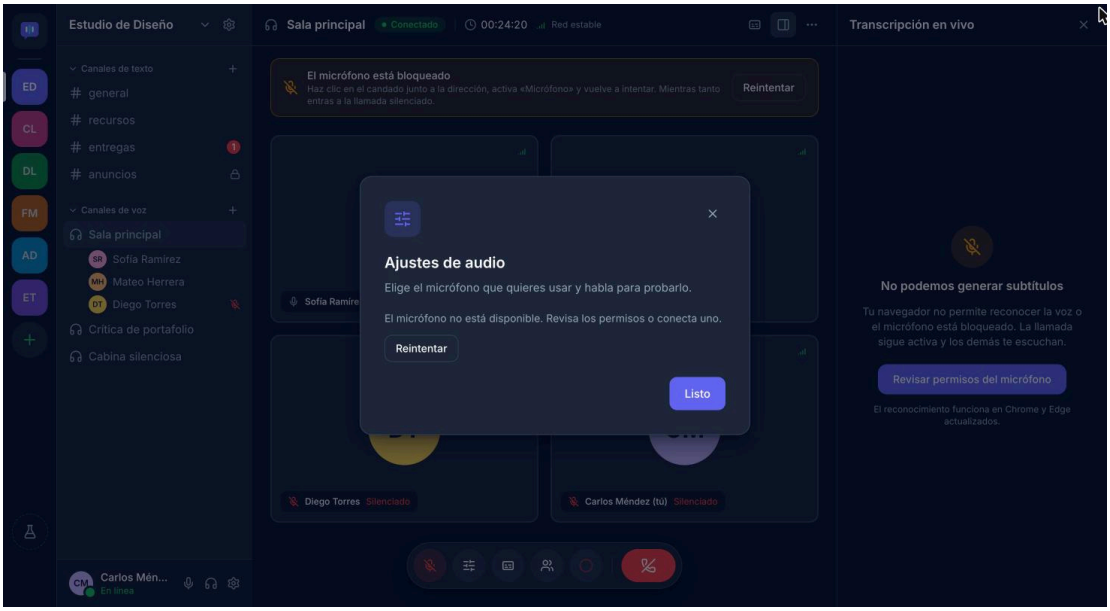


Figura 27. *Ajustes de audio sin permiso del micrófono.*

4.6 Historial y grabaciones

Aquí queda lo que se habló en los canales de voz: cada sesión con su transcripción y, si todos aceptaron, su grabación.

17 Historial de sesiones `/app/s/:serverId/sesiones`

Tabla con canal, fecha, duración, participantes, número de segmentos transcritos y estado de la grabación (grabada, procesando o sin grabación). Tiene buscador, filtros por canal, fecha y grabación, y paginación.

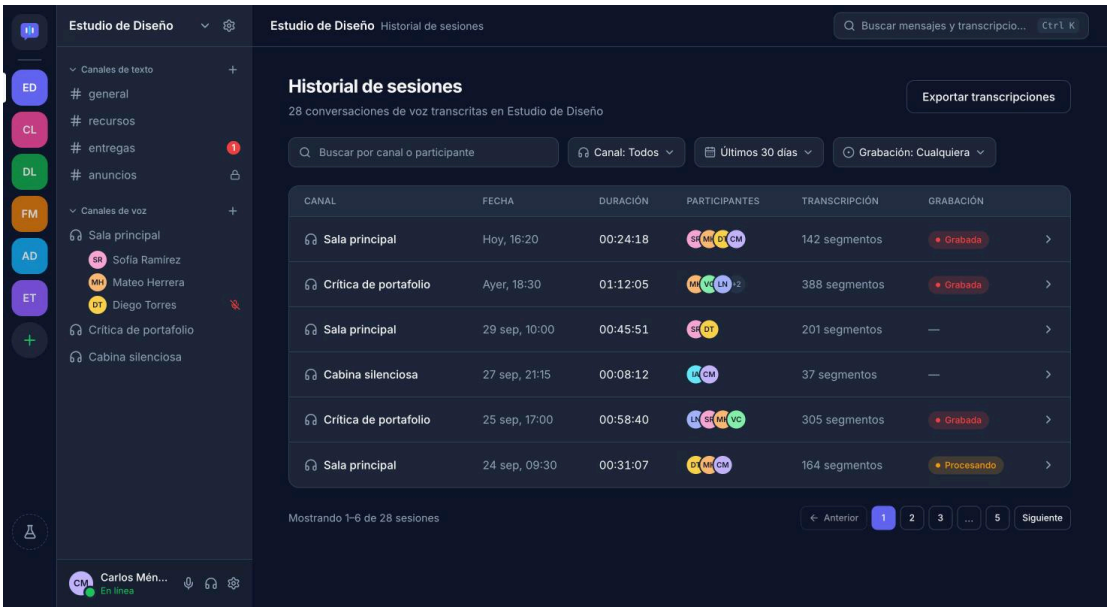


Figura 28. *Historial de sesiones de voz.*

18 Sesión — transcripción `/app/s/:serverId/sesiones/:sessionId`

Encabezado con canal, fecha, duración y participantes, y una columna con cuánto habló cada persona. La transcripción muestra cada intervención con hora y autor, y un buscador interno que resalta las coincidencias sin importar acentos («1 de 3», con flechas para recorrerlas). Al seleccionar una intervención aparece «Reproducir desde aquí».



Figura 29. *Detalle de sesión: transcripción con búsqueda.*

19 Sesión — grabación

Reproductor con forma de onda (clic para saltar), adelantar y retroceder 10 segundos, volumen y velocidad, y la lista de intervenciones que marca cuál se está reproduciendo. El audio es simulado.



Figura 30. *Detalle de sesión: reproductor de la grabación.*

20 Confirmar eliminación

Desde el menú de la grabación, «Eliminar grabación» pide confirmación. Al aceptar se muestra una notificación; la grabación todavía no se borra porque su servicio no forma parte del CRUD de esta entrega.

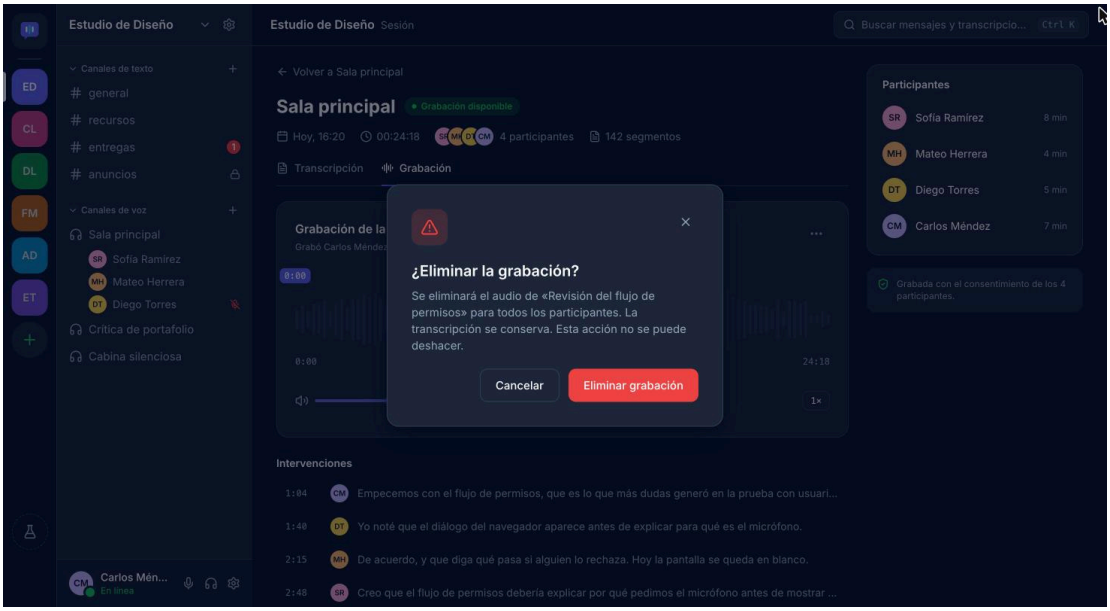


Figura 31. *Confirmación para eliminar la grabación.*



Figura 32. *Notificación después de confirmar la eliminación.*

21 Grabaciones `/app/s/:serverId/grabaciones`

Pendiente: la ruta y su guardia ya existen, pero la pantalla sigue en construcción y muestra el aviso «Por construir».

4.7 Búsqueda global

Una sola búsqueda encuentra tanto mensajes de los canales de texto como frases dichas en las llamadas transcritas. Se abre con Ctrl K desde cualquier pantalla.

22 Búsqueda global `/app/buscar?q=`

La consulta se guarda en la URL y se actualiza con debounce mientras se escribe. Hay filtros por canal, participante y fecha, y pestañas Todo, Mensajes y Transcripciones con su conteo. Cada resultado indica su tipo, ubicación y autor, y resalta los términos. Se navega con las flechas y Enter, y «Abrir en contexto» lleva a la transcripción con la palabra ya resaltada.

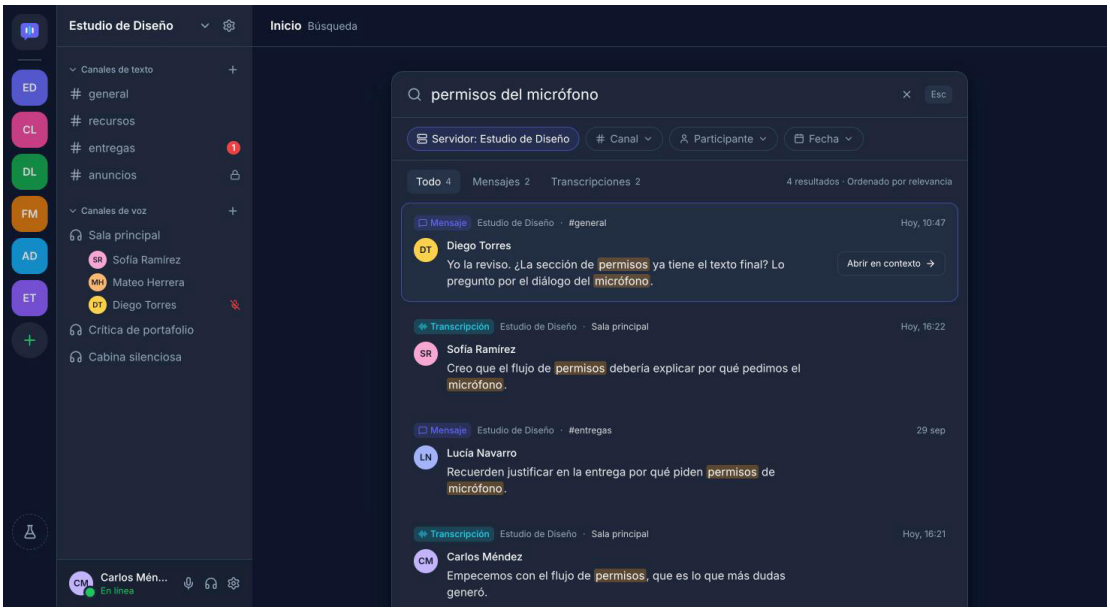


Figura 33. *Búsqueda global con resultados resaltados.*

23 Búsqueda — sin resultados

Cuando no hay coincidencias se muestra «No encontramos …» con una sugerencia para cambiar la búsqueda; si son los filtros los que dejan la lista vacía, se ofrece quitarlos.

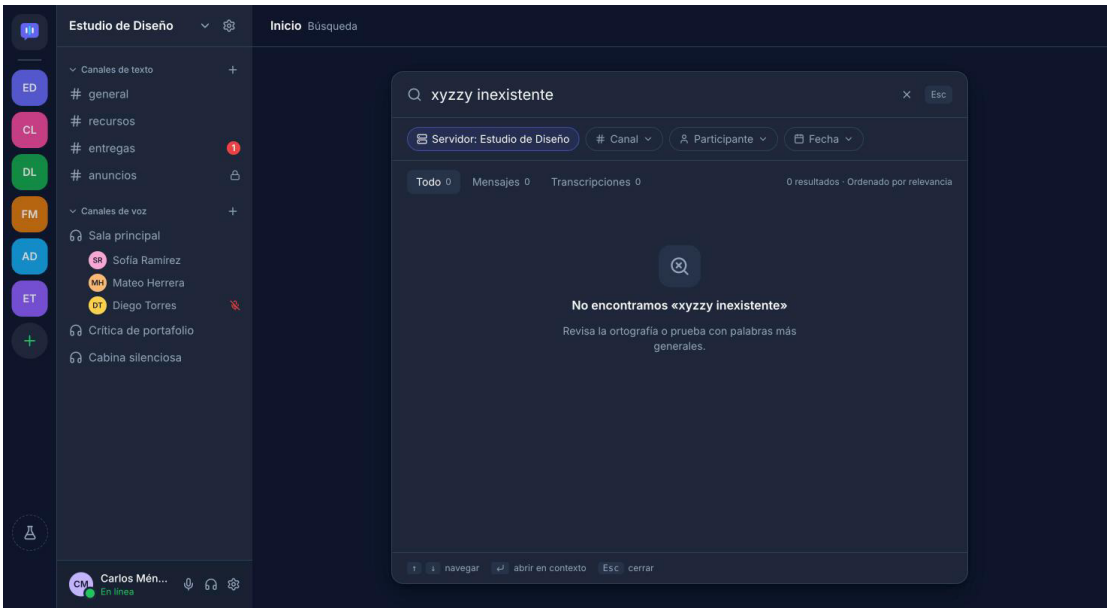


Figura 34. *Búsqueda sin resultados.*

4.8 Configuración del servidor y CRUD de canales

Solo el propietario y los administradores pueden abrirla: está protegida por RoleGuard, y el engrane del servidor se oculta con `<Can>` para los demás. Se navega con un menú lateral y se cierra con Esc.

24 Información general `/app/s/:serverId/ajustes`

Formulario con la imagen del servidor (subir o quitar), el nombre con contador de caracteres y la descripción. Al modificar algo aparece «Tienes cambios sin guardar» con las opciones de descartar o guardar.

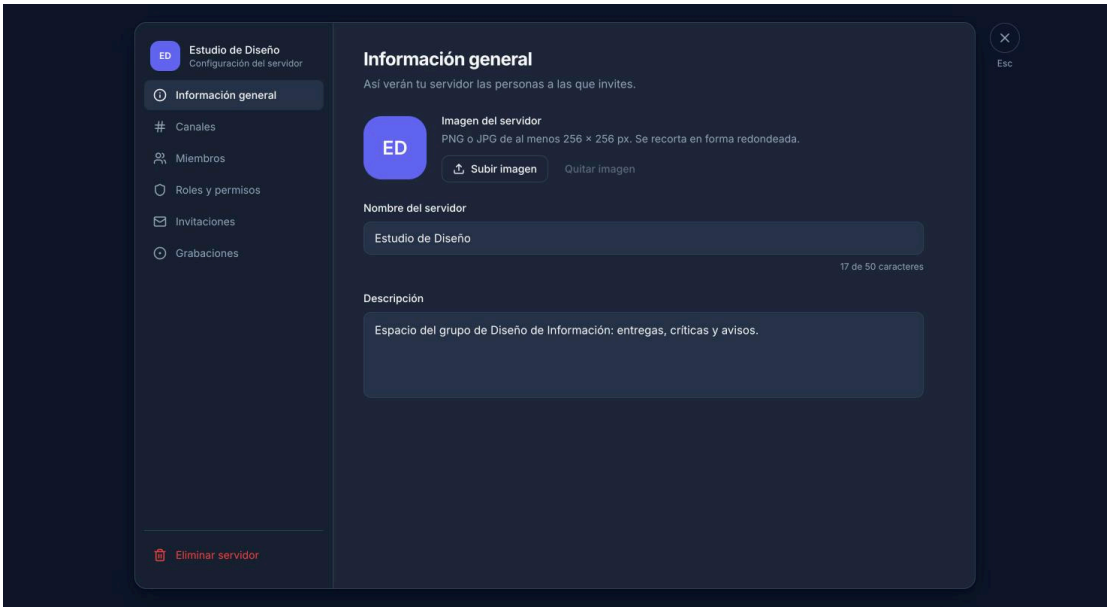


Figura 35. *Configuración: información general.*

25 Canales — CRUD completo `/ajustes/canales`

Los canales son la entidad principal con **CRUD completo** en esta entrega. Todas las operaciones pasan por `channelService` y los cambios se reflejan en la barra lateral del servidor.

Listar. Tabla de canales de texto y de voz con su tipo y acceso, y botones para editar o eliminar. Maneja carga («Cargando canales…», figura 2), error con «Reintentar» y vacío («Este servidor todavía no tiene canales»).

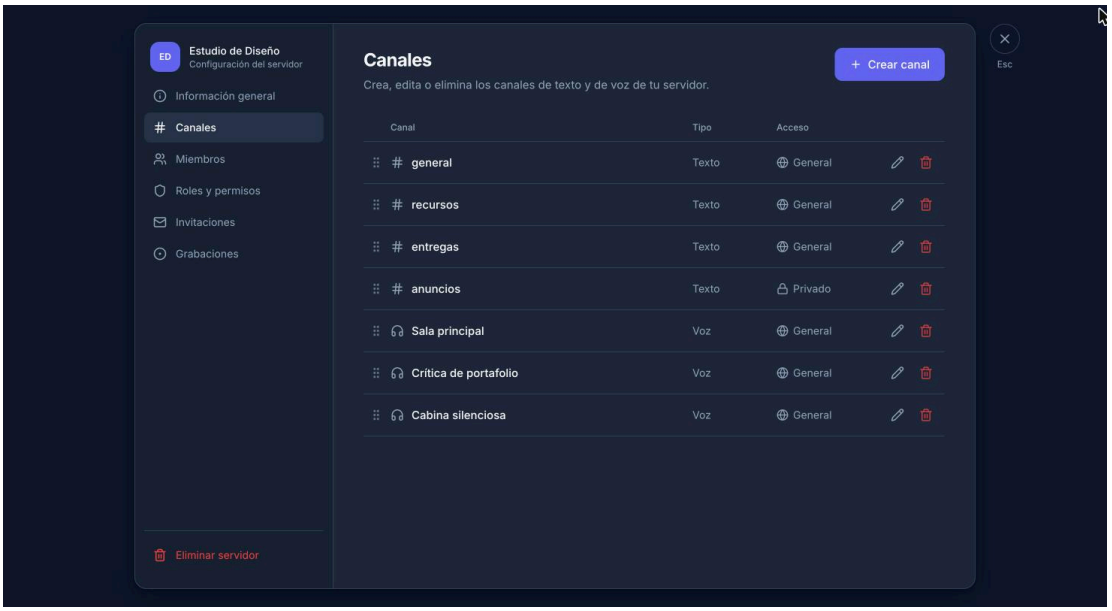


Figura 36. *Listado de canales.*

Ver detalle. Al hacer clic en el nombre de un canal se abre su detalle, que se pide con `channelService.getById`: nombre, tipo, acceso, descripción e identificador, con accesos para abrir el canal, editarlo o eliminarlo.

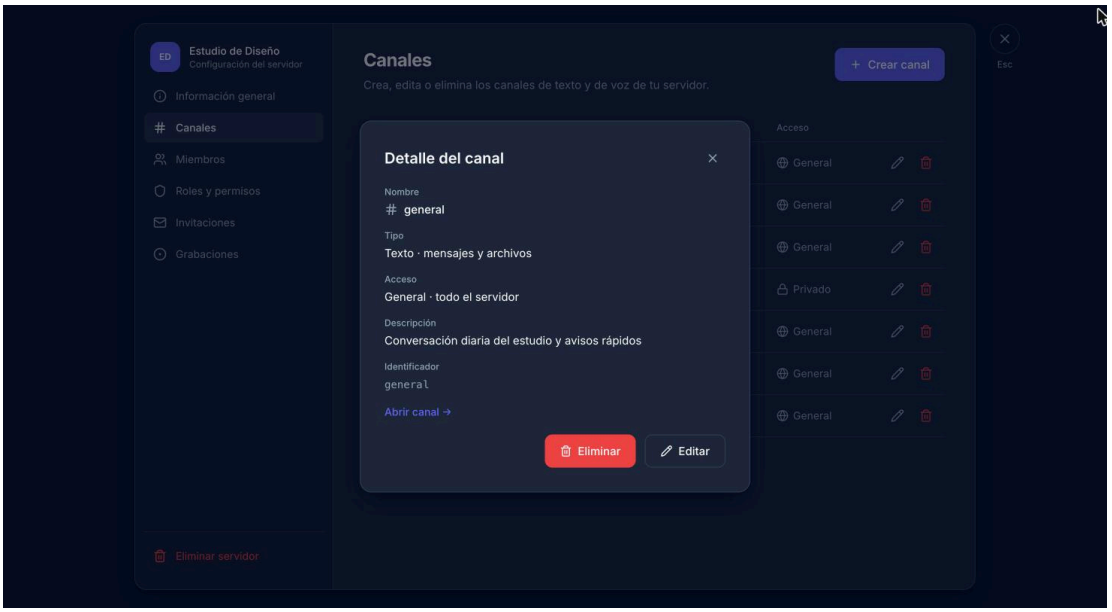


Figura 37. *Nuevo: detalle de un canal.*

Crear. «Crear canal» abre el formulario con nombre, tipo (texto o voz), descripción y acceso (general o privado con roles). Las reglas de la sección 3.4 se muestran debajo de cada campo. Si el nombre ya existe, el servicio responde 409 y el error aparece en el campo. Mientras se guarda, el botón muestra «Guardando…» y el formulario se desactiva.

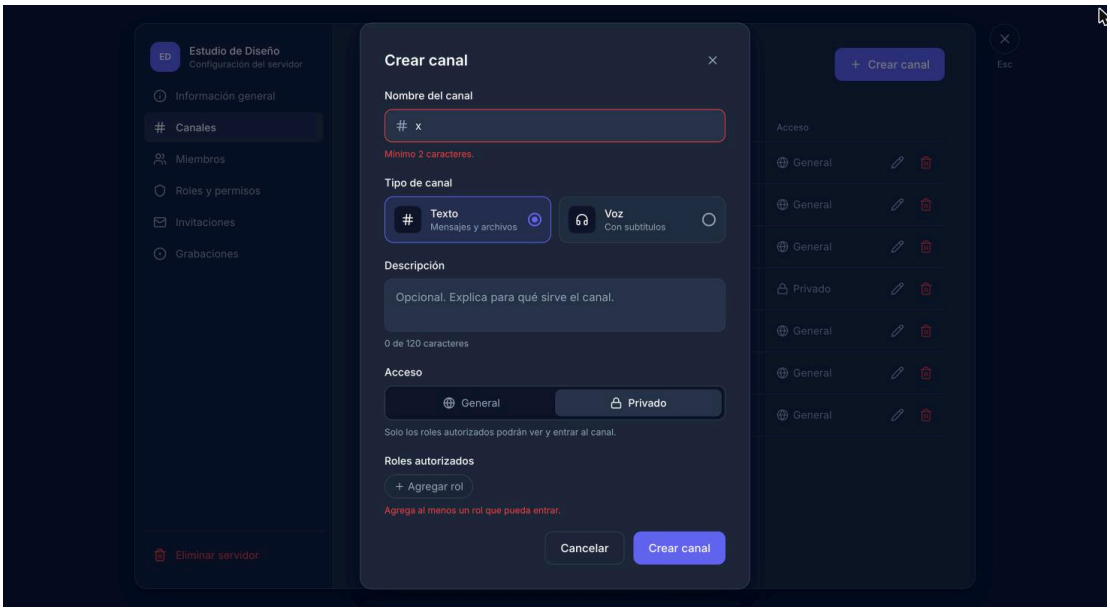


Figura 38. *Nuevo: errores de validación al crear un canal.*

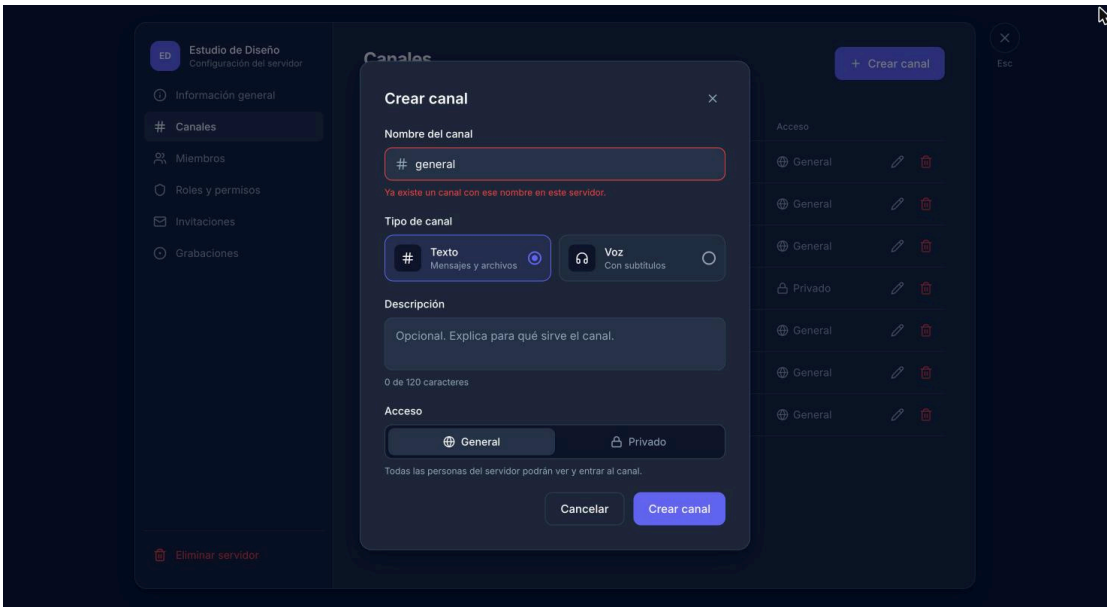


Figura 39. *Nuevo: el servicio rechaza un nombre repetido.*

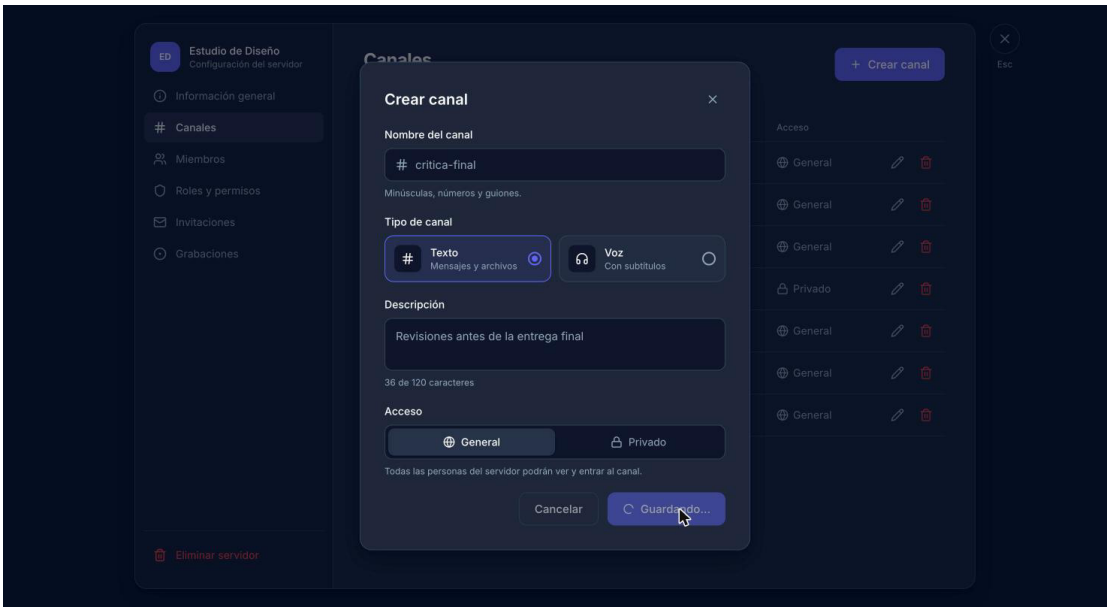


Figura 40. *Nuevo: formulario enviando, con el botón desactivado.*

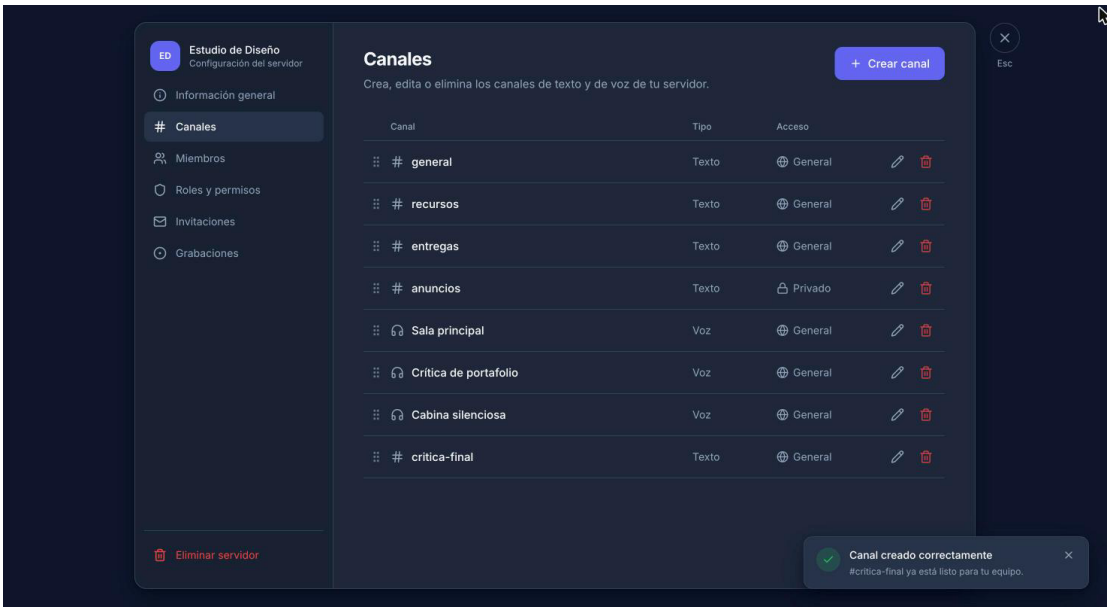


Figura 41. *Nuevo: canal creado y notificación de éxito.*

Editar. El lápiz abre el mismo formulario con los datos del canal y el botón «Guardar cambios». Aplica las mismas validaciones.

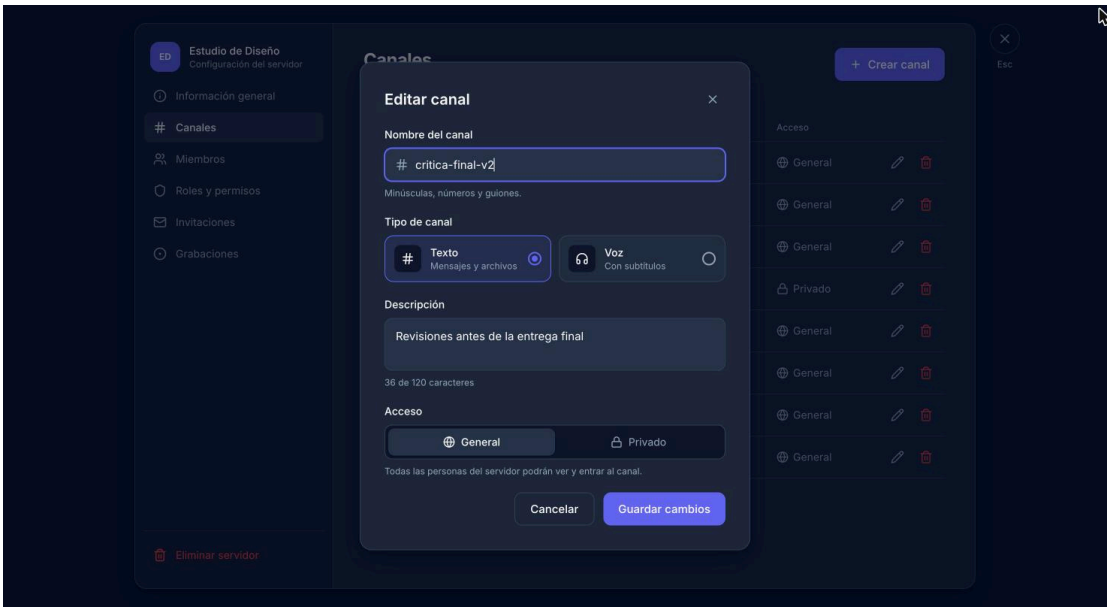


Figura 42. *Nuevo: edición de un canal.*

Eliminar con confirmación. El bote de basura pide confirmar «¿Eliminar #canal?». Al aceptar, el canal se elimina, la lista se actualiza y aparece «Canal eliminado». Si algo falla, el error se muestra con `manejarError`.

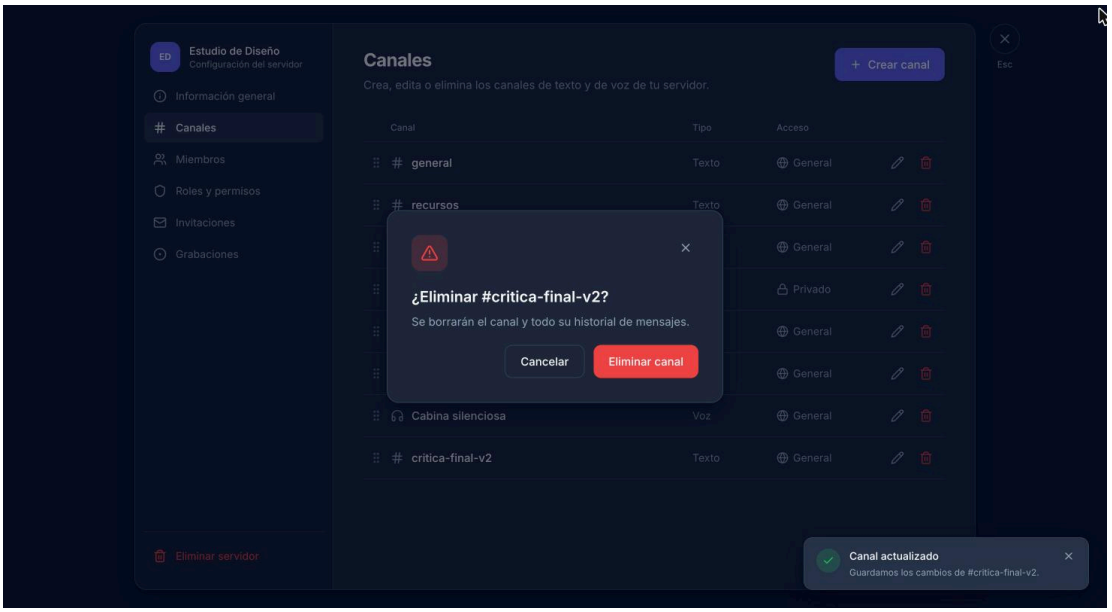


Figura 43. *Nuevo: confirmación para eliminar un canal.*

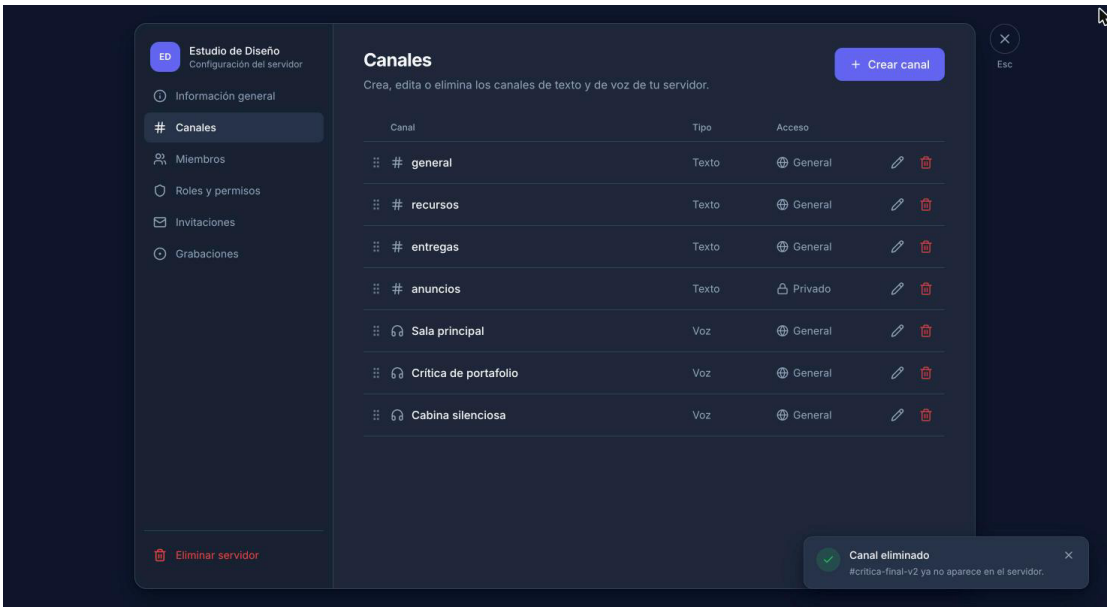


Figura 44. *Nuevo: canal eliminado y notificación.*

26 Miembros `/ajustes/miembros`

Tabla de miembros con correo, rol y estado. El propietario puede cambiar roles, y «Retirar» abre la confirmación «¿Retirar a Andrés Lozano?». Al aceptar solo se muestra una notificación.

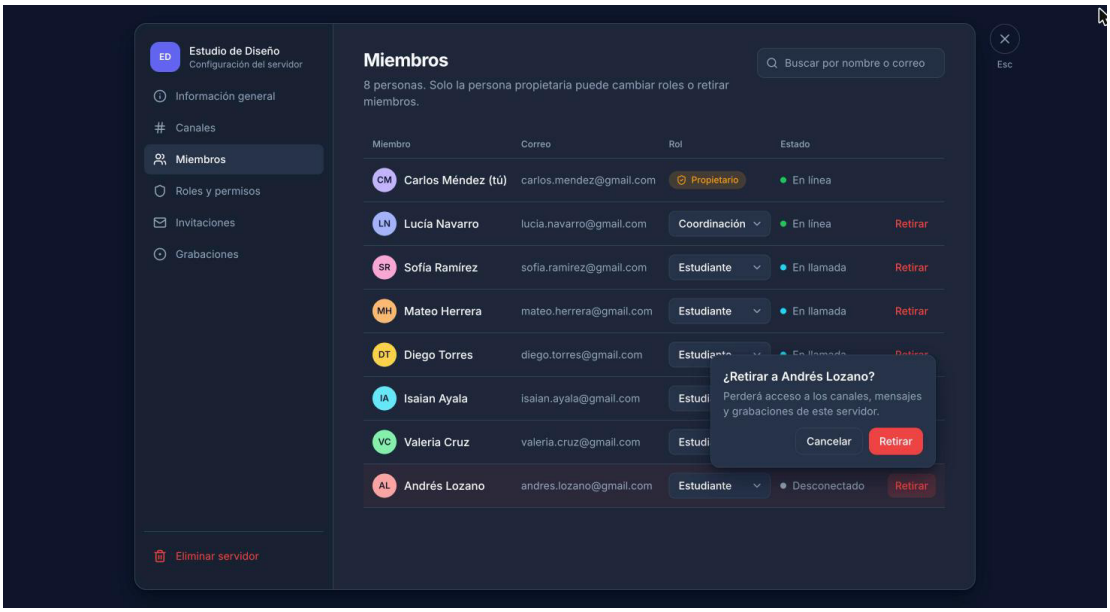


Figura 45. *Configuración: miembros con la confirmación para retirar.*

27 Eliminar servidor `/ajustes/eliminar`

Explica las consecuencias y pide escribir el nombre exacto del servidor para habilitar el botón de eliminar. Tampoco borra nada.

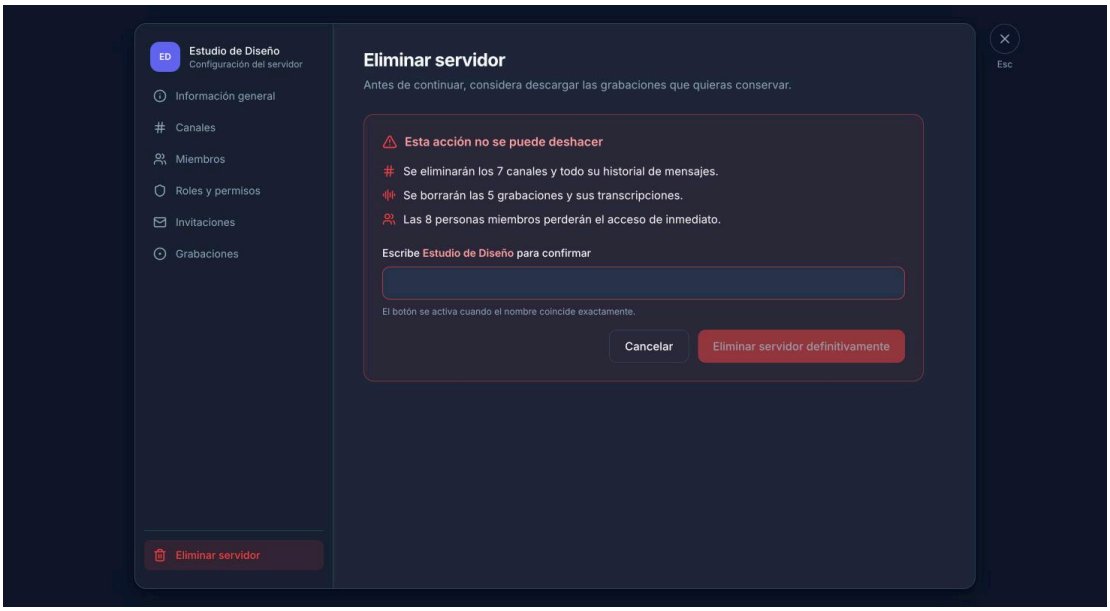


Figura 46. *Configuración: eliminar servidor.*

403 Acceso denegado

Si alguien con rol de miembro intenta abrir la configuración, la guardia lo redirige a esta vista.

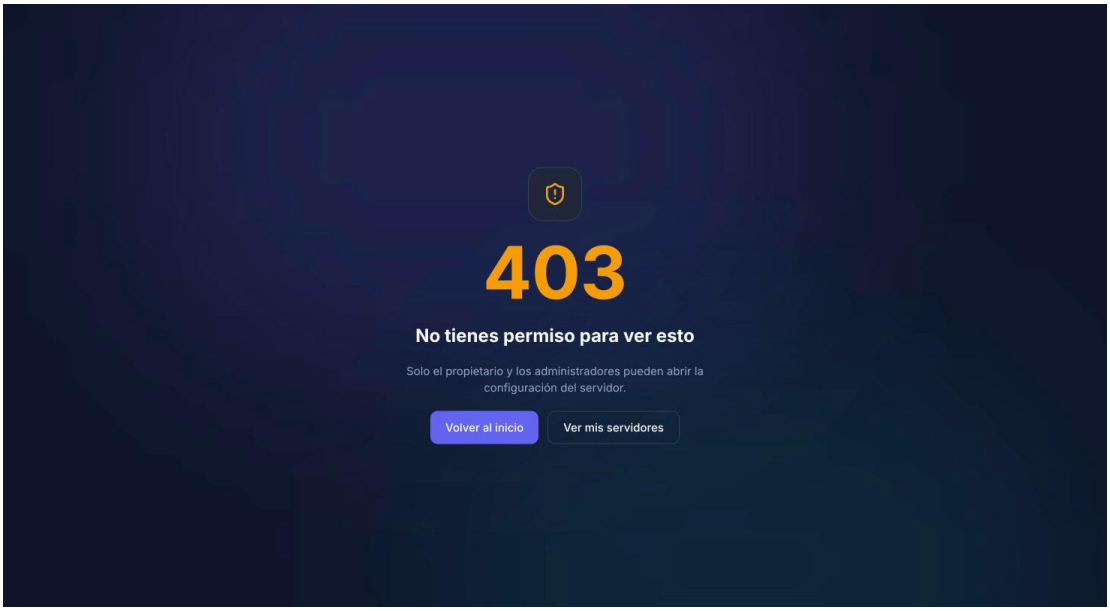


Figura 47. *Acceso denegado por la guardia de rol.*

4.9 Cuenta

28 Perfil `/app/perfil`

La tarjeta del usuario muestra su avatar con presencia, nombre, correo de Google, fecha de ingreso, número de servidores, horas en llamadas y el botón para cerrar sesión (con confirmación). El formulario

permite editar el nombre visible, el nombre de usuario y el estado, y ajustar las preferencias de subtítulos y de notificaciones. Si hay cambios sin guardar se avisa; al guardar, el nombre se actualiza en la barra lateral.

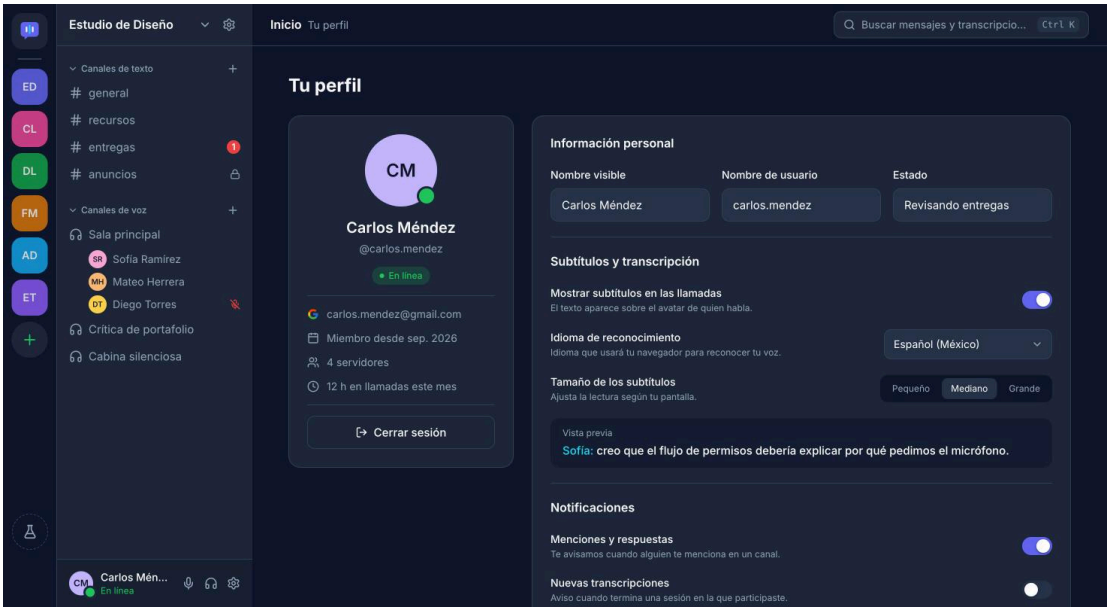


Figura 48. *Perfil del usuario.*

5. Mensajes de confirmación, notificaciones y errores

Las confirmaciones y las notificaciones son globales: cualquier vista las abre desde `uiStore` con `confirm()` y `notify()`. Los errores de la API pasan por `manejarError()` o se muestran en el campo que falló. Desde esta entrega, confirmar la eliminación de un canal sí borra el registro; las demás confirmaciones siguen siendo demostrativas.

Tipo	Ejemplo	Dónde aparece
Confirmación	«¿Eliminar #canal?» — Cancelar / Eliminar canal (elimina el registro)	25 Canales
Confirmación	«¿Eliminar la grabación?» — Cancelar / Eliminar grabación	20 Detalle de sesión
Confirmación	«¿Retirar a Andrés Lozano?» — Cancelar / Retirar	26 Miembros
Confirmación	Eliminar el servidor escribiendo su nombre; «¿Cerrar sesión?»; «¿Detener la grabación?»	27, 28, 16
Notificación de éxito	«Canal creado correctamente», «Canal actualizado», «Canal eliminado», «Servidor creado correctamente», «Te uniste a …», «Hola de nuevo, Carlos»	25, 09, 08, 02
Notificación informativa	«Enlace de invitación copiado»	09
Notificación de error	`manejarError`: «No se pudo crear el canal», «No se pudo eliminar el canal», «No se pudo subir la imagen»	25, 07, 24
Error en el campo	«Ya existe un canal con ese nombre», «Ese nombre de usuario ya está en uso», «El enlace no es válido o ya expiró»	25, 04, 08
Error del formulario	«Correo o contraseña incorrectos»	02
Aviso en pantalla	«Intentando reconectar…», «El micrófono está bloqueado», «Cargando canales…», «No encontramos …»	11, 13, 25, 23

6. Repositorio del proyecto

URL del repositorio: **https://github.com/carloso222/TalkWith**

Para ejecutarlo se necesita Node.js 20 o más reciente:

```
git clone https://github.com/carloso222/TalkWith.git
cd TalkWith
npm install
npm run dev
```

La aplicación abre en http://localhost:5173. El README del repositorio detalla la estructura, las rutas y el reparto de pantallas entre los integrantes. Datos para probar esta entrega:

- Inicio de sesión con correo: `carlos.mendez@gmail.com` y contraseña `talkwith123`. También se puede entrar con «Continuar con Google» (simulado).
- Invitación para unirse a un servidor: `talkwith.app/i/arte-digital-5MZ8`.
- Los cambios se guardan en el navegador. Para volver a los datos de ejemplo, borra el almacenamiento local del sitio.
- Para ver los estados del canal de voz sin depender del micrófono: `?demo=sin-subtitulos`, `?demo=solicitud` o `?demo=grabando` en la URL de Sala principal.

Reparto de pantallas y tareas

Integrante	Trabajo
Carlos Olmedo	02–03 Inicio de sesión, 04 Registro, 18–20 Detalle de sesión, 22–23 Búsqueda, 28 Perfil; estructura base del proyecto, rutas y guardias.
Diego Gómez	01 Landing, 05 Inicio, 12–16 Canal de voz y grabación (subtítulos, transcripción en vivo y consentimiento), 29 Página no encontrada; layout y componentes base. Pendiente: 21 Grabaciones.
Isaian Ayala	06 Tus servidores, 07–09 Crear servidor y notificaciones, 08 Unirse, 10–11 Canal de texto, 17 Historial, 24–27 Configuración. En la Entrega 3: capa de errores y validaciones, CRUD completo de canales, formularios validados de inicio de sesión y registro, crear servidores y unirse con invitación, y acceso al micrófono en los canales de voz.